

Should You Even Train a Neural Network? A Forecasting-Based Early Rejection Strategy under Budget Constraints

Santiago Ramírez Castañeda

Universidad de La Sabana

Faculty of Engineering: Maestría en Analítica Aplicada

Félix Mohr

10th of October 2025

Should You Even Train a Neural Network? A Forecasting-Based Early Rejection Strategy under Budget Constraints

Abstract

Neural networks have achieved remarkable performance across diverse tasks, yet their design and training remain computationally demanding due to vast architectural search spaces and long convergence times. This thesis investigates whether early-training signals can reliably predict when neural architectures are worth pursuing relative to strong non-neural baselines. Building on this premise, we propose *ShouldYouEvenNN*, a decision-oriented framework that reframes Neural Architecture Search (NAS) as a probabilistic decision problem rather than a purely optimization task. The framework leverages early learning dynamics and lightweight forecasting to estimate the likelihood that a neural candidate will surpass non-neural models, integrating this evidence into an expected-utility formulation that weighs potential performance gains against projected computational cost. Through this mechanism, *ShouldYouEvenNN* selectively continues training only when early evidence suggests a positive expected return, effectively governing when neural exploration is justified. Empirical evaluations across 71 OpenML datasets demonstrate that the framework can forecast and select architectures approaching oracle-level performance while remaining competitive with ensemble-based baselines at a fraction of the computational cost. These findings indicate that early-training information, when modeled probabilistically, provides a robust and reproducible foundation for fast, cost-aware neural architecture search.

Introduction

Neural networks have transformed modern machine learning, achieving state-of-the-art performance across diverse domains such as vision, language, and forecasting. Yet their design and training remain expensive and limiting. The vastness of possible architectures, hyperparameters, and learning dynamics makes neural exploration a computational gamble—especially when each evaluation demands full convergence to reveal its true potential.

Paradoxically, in structured or tabular domains—the setting of most practical data analytics—non-neural methods still dominate (Shwartz-Ziv & Armon, 2022). Ensemble-based models such as Gradient Boosted Trees and Random Forests routinely outperform neural networks under comparable optimization budgets, offering strong generalization with minimal tuning and far lower computational cost (Borisov et al., 2022; Grinsztajn et al., 2022; Mohr & van Rijn, 2023). This

persistent asymmetry raises a fundamental and largely unaddressed question: *when is training a neural network actually worth it?*

Automated Machine Learning (AutoML) frameworks have made great strides toward automation and efficiency. They identify strong baseline models through algorithm selection and hyperparameter optimization, often without requiring deep expertise (Elsken et al., 2019; Mohr & Wever, 2023). However, while AutoML systems streamline the discovery of high-performing non-neural configurations, Neural Architecture Search (NAS) extends this automation to deep models—frequently at the expense of massive computational budgets. Based on our initial review, most NAS research focuses on accelerating search within the neural family, rather than determining whether neural architectures are even competitive for the problem at hand.

Recent work has shown that early training dynamics—partial learning curves, gradient signals, or batch-level metrics—can reveal predictive information about a model’s eventual performance (Domhan et al., n.d.; Egele et al., 2024; Ren et al., 2022). These findings suggest that evaluating a candidate’s promise does not necessarily require full training. What remains missing is a principled mechanism to *act* on this information: a decision rule that weighs early neural evidence against efficient non-neural baselines.

This thesis introduces *ShouldYouEvenNN*, a forecasting-driven decision framework designed to answer precisely that question. Instead of treating NAS as a blind search for optimal architectures, *ShouldYouEvenNN* formulates it as a rational decision process under computational uncertainty. Each neural candidate is allocated a uniform and limited amount of training effort, from which early validation signals are extrapolated to forecast long-term accuracy. These forecasts are then evaluated probabilistically against a reference baseline through an expected-utility criterion that quantifies whether continued neural training is justified.

By transforming early learning behavior into actionable evidence, *ShouldYouEvenNN* bridges the gap between AutoML efficiency and NAS exploration. The remainder of this work situates *ShouldYouEvenNN* within the broader landscape of automated model design and selection. The next section reviews the evolution of AutoML systems, emphasizing how they streamline prepro-

cessing, algorithm selection, and hyperparameter optimization to identify competitive non-neural baselines efficiently. It then contrasts these approaches with NAS methods, which aim to automate neural design but remain constrained by high computational costs. Finally, it surveys recent advances in early-termination and learning-curve forecasting strategies that seek to reduce such costs—developments that directly motivate the decision-oriented formulation adopted by *ShouldYouEvenNN*.

Background

Background and Related Work

Automated Machine Learning and Model Selection

AutoML aims to make advanced modeling broadly accessible by automating the stages of data preprocessing, model selection, and hyperparameter optimization. By formalizing these steps as a joint optimization problem, AutoML systems remove much of the trial-and-error traditionally required in machine learning workflows (Baratchi et al., 2024). A central theoretical formulation within AutoML is the *Combined Algorithm Selection and Hyperparameter Optimization* (CASH) problem (Thornton et al., 2013), defined as

$$A^*, \lambda^* \in \arg \min_{A^{(j)} \in \mathcal{A}, \lambda \in \Lambda^{(j)}} \frac{1}{k} \sum_{i=1}^k \mathcal{L}(A_{\lambda}^{(j)}, D_{\text{train}}^{(i)}, D_{\text{valid}}^{(i)}),$$

where $\mathcal{A}$ denotes the set of learning algorithms, $\Lambda^{(j)}$ their hyperparameter spaces, and $\mathcal{L}$ a validation loss function. This definition formalizes the automated search for the optimal algorithm–configuration pair (A^*, λ^*) that minimizes validation error over k folds.

Early AutoML frameworks approached the CASH problem using Bayesian optimization and ensemble strategies to traverse algorithm and hyperparameter spaces efficiently (Nguyen et al., 2022; van Hoof & Vanschoren, 2021; Z. Wang et al., 2017; Wu et al., n.d.). Subsequent generations integrated meta-learning and evolutionary search (Jurado & Vanschoren, 2022; Salehin et al., 2024), balancing exploration and exploitation to reduce runtime while maintaining accuracy. Recent work, by Mohr and Wever (2023) demonstrated that much of this complexity can be re-

duced through separable optimization: optimizing preprocessing and modeling stages independently yields comparable performance to full CASH solvers, revealing that simplicity and transparency often outperform algorithmic intricacy.

AutoML’s evolution reflects a clear principle: efficient automation can replace exhaustive experimentation without sacrificing generalization. This principle provides the foundation upon which neural automation—specifically, NAS—was later built.

Neural and Non-Neural Baselines on Tabular Data

Despite the success of neural networks in unstructured domains such as vision, text, and speech, their dominance does not extend to tabular or structured data. Comprehensive reviews and benchmarks consistently show that tree-based ensembles, particularly Gradient Boosted Decision Trees (GBDTs) and Random Forests, outperform feedforward neural networks on the majority of tabular datasets (Borisov et al., 2022; Grinsztajn et al., 2022; Mohr & van Rijn, 2023). The reason is both architectural and statistical: GBDTs naturally handle heterogeneous feature scales, categorical variables, and missing values without extensive preprocessing, whereas neural networks require larger datasets, hyperparameter sensitivity, and extensive regularization to achieve comparable results.

Nonetheless, this superiority is not absolute. McElfresh et al. (2023) showed that deep networks can outperform GBDTs in high-dimensional and interaction-heavy datasets, provided sufficient data and regularization. Gardner et al. (2022) found that ensembles often yield better subgroup robustness, while neural networks can capture nonlinear feature hierarchies inaccessible to additive models. Also, Gorishniy et al. (2021) found no universally superior solution when testing GBDTs versus models such as ResNet or Transformers. The tension between flexibility and efficiency persists: neural models can, in principle, surpass non-neural ones, but only under favorable conditions that justify their cost. This tension motivates the central question underlying this work.

Neural Architecture Search and the Problem of Efficiency

NAS **extends** AutoML from algorithm selection to architecture design. Given a neural architecture A parameterized by weights θ , the NAS objective seeks to minimize validation loss:

$$\min_{\theta} \mathcal{V}(A, \theta^*(A)) \quad \text{subject to} \quad \theta^*(A) = \arg \min_{\theta} \mathcal{L}(A, \theta),$$


where $\mathcal{V}$ denotes validation performance and $\mathcal{L}$ the training loss. This highlights NAS's inherent cost: evaluating $\mathcal{V}$ requires full training for every candidate A .

NAS approaches have relied on reinforcement learning and evolutionary algorithms (Mendoza et al., 2016; Zoph & Le, 2016), but computational demands were immense—training tens of thousands of networks from scratch on massive datasets. Later works introduced weight sharing and one-shot models (Jin et al., 2019; Lu et al., 2024), as well as differentiable search spaces (Liu et al., 2024; W. Wang & Li, 2025), drastically reducing redundant computation. Hybrid methods such as Lin et al. (2024), Sakamoto et al. (2023), and Zheng et al. (2021) further improved exploration through multi-armed bandits and task transfer.

Still, scalability and reproducibility remain open issues (Mohr & van Rijn, 2022). Even efficient NAS systems are often constrained by the cost of evaluating candidates to convergence, making them impractical for small or medium-sized problems. Ren et al. (2022) and Garcia-Garcia et al. (2025) proposed progressively expanding training budgets to reduce wasted computation, while Jurado and Vanschoren (2022) introduced multi-fidelity strategies that evaluate partial models before committing additional resources. Despite these advances, most approaches still assume that some form of full training will ultimately be completed. The missing element is a mechanism to determine whether such training should even begin.

Early-Training Signals and Lightweight Evaluation

Research has shown that early training signals (e.g recorded after only a few epochs or even mini-batches) can be predictive of final model quality. Learning-curve extrapolation methods such as Domhan et al. (n.d.) and Swersky et al. (2014) fit exponential, logistic, or rational functions to

partial validation trajectories, estimating their asymptotic performance. Subsequent work by White et al. (2021), Wistuba and Pedapati (2020), and Yan et al. (2021) expanded this approach to NAS benchmarks, showing that forecast accuracy correlates strongly with final validation outcomes.

Alternative methods exploit static or gradient-based indicators: Wen et al. (2019) introduced zero-cost proxies based on Jacobian conditioning, while Ru et al. (n.d.) used gradient norms to predict overfitting potential. Probabilistic and Bayesian approaches (Adriaensen et al., 2023; Wei et al., 2020) combine these signals into uncertainty-aware forecasts, accelerating NAS by pruning underperforming candidates early. Parallel research in AutoML applied similar ideas to time-constrained model search, using partial evaluations and meta-surrogates to approximate performance across pipelines (Aach et al., 2023; Guan et al., 2025; Solís-Martín et al., 2025).

The idea of halting or downscaling unpromising runs has also been applied to hyperparameter optimization. Egele et al. (2023, 2024) demonstrated that one-epoch performance can approximate the outcome of extensive search, with negligible accuracy loss and substantial time savings. Such findings suggest that early metrics are not merely training diagnostics but informative variables—signals from which future performance can be inferred.

Forecasting and Decision-Making in NAS

As seen, forecasting has become an integral part of efficiency-driven NAS. Bayesian curve-fitting (Adriaensen et al., 2023), Gaussian-process surrogates, and polynomial extrapolations (Mebirouk et al., 2024; Patel et al., 2024) enable fast ranking of candidates based on partial data. W. Wang and Li (2025) and Nandal et al. (2025) introduced uncertainty quantification into this process, allowing NAS algorithms to balance exploration and exploitation adaptively. However, even the most advanced forecasters typically serve as auxiliary tools for ranking, not as explicit decision-makers. The question of whether neural exploration is worth continuing at all—remains largely heuristic.

Problem Definition and Motivation

The literature reviewed above reveals a clear imbalance in how automation has evolved across learning paradigms. While AutoML has successfully optimized model selection and hyperparam-

eters across non-neural families, NAS remains heavily focused on structural optimization rather than strategic decision-making. That is, NAS seeks the best neural architecture *assuming* neural models are worth training, whereas AutoML frameworks often demonstrate that simpler, non-neural models suffice for most real-world problems (Borisov et al., 2022; Grinsztajn et al., 2022; Mohr & van Rijn, 2023). This assumption of neural worthiness produces a systematic inefficiency: NAS frameworks spend vast computational budgets exploring architectures that, for many datasets, cannot outperform ensemble-based baselines.

The challenge, therefore, is not merely to accelerate NAS, but to determine when neural training is rational in the first place. Multi-fidelity and early-termination techniques (Egele et al., 2023, 2024; Garcia-Garcia et al., 2025; Jurado & Vanschoren, 2022; Ren et al., 2022) mitigate cost by pruning unpromising runs; yet these strategies remain reactive—they halt low-performing candidates but do not decide whether neural exploration itself is justified. The field lacks a unified mechanism for comparing early neural progress against established non-neural standards to decide whether additional neural optimization is worthwhile.

This gap motivates a reframing of NAS as a *decision problem under uncertainty*. Rather than searching exhaustively within a neural space, the goal is to use partial training evidence to estimate the probability that neural models can surpass non-neural baselines, and to weigh that probability against the expected computational cost of continuing. This view treats model training not as a guaranteed investment but as a stochastic process whose continuation should depend on its expected return.

Let $\mathcal{V}_{\text{NN}}$ denote the forecasted validation performance of a neural model, and $\mathcal{V}_{\text{base}}$ the performance of a strong non-neural baseline such as a GB ensemble. Given early training observations, one can estimate the probability

$$p = P(\mathcal{V}_{\text{NN}} > \mathcal{V}_{\text{base}} \mid \text{early evidence}),$$

representing the likelihood that continued neural training will yield superior performance. Defining $\Delta = \mathbb{E}[\mathcal{V}_{\text{NN}} - \mathcal{V}_{\text{base}}]$ as the expected gain and C as the projected computational cost of continuation,

a rational decision mechanism seeks to maximize the expected utility

$$\boxed{\equiv} \quad U = w \cdot (p \cdot \Delta) - \lambda \cdot C,$$

where w is an evidence-weighting factor that increases with the reliability of early forecasts, and λ is a cost-scaling parameter controlling the tolerance for computational expenditure. The decision rule is then straightforward: continue training if $U > 0$, otherwise terminate exploration and favor non-neural alternatives.

This formulation establishes a probabilistic and cost-aware framework that transforms early-learning information into actionable guidance. Instead of focusing exclusively on architecture search or hyperparameter tuning, *ShouldYouEvenNN* introduces a decision layer that asks, given limited evidence, whether investing additional computation in neural training is expected to pay off. This perspective directly integrates insights from AutoML optimization (Baratchi et al., 2024; Mohr & Wever, 2023; Thornton et al., 2013), NAS efficiency research (Lin et al., 2024; Liu et al., 2024; Mendoza et al., 2016; W. Wang & Li, 2025; Zoph & Le, 2016), and early-signal forecasting studies (Adriaensen et al., 2023; Domhan et al., n.d.; Wei et al., 2020; White et al., 2021).

In this sense, *ShouldYouEvenNN* does not compete with NAS—it governs it. It serves as a probabilistic decision mechanism capable of halting neural exploration when the evidence suggests that the expected payoff no longer justifies the computational cost. This conceptual shift transforms neural architecture search from an open-ended optimization procedure into a disciplined, empirically-driven decision process, aligning model discovery with the principles of rational resource allocation.

Research Question

The core question behind this research is: *Can early-training signals and lightweight evaluation strategies be effectively used to predict whether neural network architectures will achieve competitive performance compared to strong non-neural models, enabling a fast and resource-efficient NAS framework?*

Research Objectives

General Objective

To develop a fast and resource-efficient NAS framework capable of determining whether neural networks are likely to yield competitive performance compared to strong non-neural models, by leveraging early training information and lightweight evaluation strategies.

Specific Objectives

- Develop a lightweight mechanism that leverages early training behavior to estimate the **likelihood** that a neural network will outperform established non-neural baselines, enabling informed continuation or termination of neural exploration.
- Design and Implement a simple decision-oriented NAS framework that combines probabilistic scoring with evolutionary refinement, allowing progressive improvement of candidate architectures under constrained time and data budgets while avoiding unnecessary neural training.
- Empirically validate the framework by benchmarking its capacity to identify tasks where neural networks provide a measurable advantage over non-neural models, evaluating both predictive performance and computational efficiency.

Methodology



Data Preparation

All datasets were retrieved from the OpenML repository and preprocessed through a deterministic pipeline. Categorical variables are label-encoded and numerical ones standardized using the training split as reference. Data are divided into training, validation, and test sets with an 80/10/10 ratio, preserving class **balance** through stratified sampling. Rare or unseen target classes are mapped to a shared `__other__` label for consistent encoding across splits. Missing values are imputed by mean or mode depending on data type, and all splits are converted to PyTorch tensors for reproducible training.

Candidate Generation and Search Space



Candidate architectures were sampled from a structured but bounded search space of multilayer perceptrons (MLPs), designed to ensure both diversity and computational feasibility. Each configuration specifies the number of layers, neurons per layer, activation functions, dropout, optimizer, and learning rate schedule.

Table 1 summarizes the default configuration.

Shape Pattern. This hyperparameter controls the overall distribution of neurons across the hidden layers, defining how the network’s capacity expands or contracts throughout its depth. Different patterns influence gradient flow, representation compression, and regularization behavior. The following configurations were used:

- **Constant:** All hidden layers contain the same number of neurons, producing a uniform capacity distribution.
- **Pyramid:** The number of neurons decreases monotonically toward deeper layers, forming a funnel-like topology.
- **Inverted:** The opposite of the Pyramid pattern—deeper layers become wider.
- **Hourglass:** The network first compresses and then re-expands, mimicking autoencoder-like bottlenecks.
- **Triangular:** The layer widths increase up to a midpoint and then decrease symmetrically, creating a single capacity peak.
- **Irregular:** Neuron counts are sampled randomly within the allowed range, producing non-monotonic shapes. This introduces structural diversity that can improve evolutionary search exploration.

Each candidate is instantiated deterministically under a shared random context to guarantee full reproducibility. Initial populations are composed of purely random architectures, while later ones

Table 1*Search space configuration for neural candidates.*

Hyperparameter	Range / Options
Hidden layers	2–25
Neurons per layer	3–100
Activations*	ReLU, LeakyReLU, Sigmoid, Tanh, ELU, GELU
Dropout	0.0–0.5 (step 0.1)
Optimizer	Adam, SGD, RMSprop, AdamW
Learning rate	10^{-4} – 10^{-1} (log-uniform)
Batch size	32–1024 (powers of two)
Weight decay	0 – 10^{-2}
Momentum (SGD only)	0.8–0.99
LR scheduler	Step, Exponential, Cosine, None
Shape pattern	Constant, Pyramid, Inverted, Hourglass, Triangular, Irregular

*Activation functions were applied uniformly across all hidden layers.

may include mutated offspring from high-performing candidates (See Evolutionary Search and Selection section).

Batch-Level Training Protocol

Each candidate is trained for *one epoch per generation* on a progressively larger subset of the training data. After every epoch, the instance budget is increased by a multiplicative growth factor (here, 1.4), which expands the effective sample seen by the model before the next validation point. This preserves the one-epoch rhythm while steadily increasing evidence, enabling early yet reliable forecasts.

At every iteration, the model performs a single forward and backward pass per batch, updating its weights and logging both training and validation metrics. Every processed batch is counted as one training unit, making progress comparable across architectures with different batch sizes or computational footprints.

Validation accuracy is evaluated at regular batch intervals. The sequence of validation metrics $\{y_t\}$ forms a partial learning curve that serves as input to the forecasting mechanism.

Forecasting from Early Training Signals

The forecasting module predicts whether a partially trained neural candidate is likely to outperform a given non-neural baseline before full training is completed. Given validation accuracies $y = \{y_1, \dots, y_t\}$ observed at batch indices $x = \{1, \dots, t\}$, the module performs the following steps:

1. **Forecast horizon.** Each curve is projected to a future batch index:

$$T_f = t_{\text{now}} + N \cdot \max(t_{\text{now}}, t_{\text{dataset}}),$$

where t_{dataset} is the number of batches per full dataset pass, and N (default $N = 5$) defines the extrapolation depth.

2. **Shape-aware curve morphing.**

Rationale for Shape-Aware Curve Morphing. Neural learning curves exhibit heterogeneous behaviors across training phases: an initial region of rapid growth, a transitional plateau, and a final stage of stabilization. Using a single extrapolation model (e.g., rational, sigmoid, or linear) cannot capture these regime shifts reliably under limited early data. To address this, the proposed shape-aware morphing mechanism combines three complementary regressors (rational, sigmoid, and linear) through adaptive weights derived from the current slope and curvature of the validation trajectory. This formulation allows smooth transitions between acceleration, saturation, and steady-state behaviors, reducing extrapolation bias and overfitting to local noise. By dynamically adjusting to the curve geometry, the morphing forecaster provides more stable and conservative accuracy predictions, improving the reliability of the subsequent probability and expected-utility estimations in the following way:



$$f_{\text{morph}}(x) = w_r f_{\text{rational}}(x) + w_s f_{\text{sigmoid}}(x) + w_l f_{\text{linear}}(x),$$

where $f_{\text{rational}}(x) = \frac{ax}{b+x}$, $f_{\text{sigmoid}}(x) = \frac{L}{1+e^{-k(x-x_0)}}$, and $f_{\text{linear}}(x) = mx + c$. The weights (w_r, w_s, w_l) are derived from the most recent slope and curvature of the validation curve, ensuring smooth

transitions between growth, plateau, and stabilization.

3. **Confidence interval.** The forecast $\hat{y}_f = f_{\text{morph}}(T_f)$ is bounded by a confidence interval derived from residual variance:

$$[\hat{y}_{\text{low}}, \hat{y}_{\text{high}}] = [\hat{y}_f - t_\alpha \sigma_y, \hat{y}_f + t_\alpha \sigma_y],$$

where σ_y^2 is the residual variance and t_α is the two-tailed critical value of the t -distribution.

4. **Probability of improvement.** To estimate how likely a candidate is to surpass the baseline, two complementary probabilities are computed. Let g be the baseline accuracy and $\hat{a}_{\text{hi}}$ the upper confidence-bound forecast.

The first term, $s(\hat{a}_{\text{hi}}, s, v)$, expresses a *conservative* confidence based on a penalized logistic transformation:

$$s(\hat{a}_{\text{hi}}, s, v) = \sigma \left(\frac{(\hat{a}_{\text{hi}} - g) - \pi(s, v)}{\tau} \right),$$

where $\sigma(\cdot)$ is the sigmoid function and $\tau = 0.05$ controls the sharpness of the transition. The adaptive penalty

$$\pi(s, v) = \lambda_s 0.1 e^{-\lambda_s \max(0, s)} + \lambda_v v$$

reduces confidence when the recent learning curve is too steep ($s > 0$) or unstable (large v).

The second term, $m(\hat{a}_{\text{hi}}, v)$, gives an *optimistic* probability under a Gaussian uncertainty model:

$$m(\hat{a}_{\text{hi}}, v) = \Pr[\mathcal{N}(\hat{a}_{\text{hi}}, \sqrt{v}) > g],$$

quantifying the chance that a normally distributed forecast exceeds g . These complementary perspectives are merged as

$$p_{\text{base}} = \alpha m + (1 - \alpha) s, \quad \alpha = 0.7,$$

and further stabilized using the latest observed validation accuracy a_{last} :

$$p = \max\{0.05, 0.6p_{\text{base}} + 0.4a_{\text{last}}\}.$$

Rationale for the Penalized Logistic Transformation. A plain sigmoid would turn any small forecast advantage ($\hat{a}_{\text{hi}} - g$) into a high probability of success, even when the learning curve is noisy or just momentarily improving. To avoid such false optimism, a penalty term $\pi(s, v)$ is added. This penalty reduces confidence when the recent slope s is too steep (suggesting the model is rising too fast to be trusted) or when the validation accuracy has high variance v (indicating unstable learning). The result is a more cautious probability estimate that rewards steady, reliable progress and prevents the system from overestimating a model’s potential too early.

Forecasting and Scoring Procedure. The entire process operates at the batch level for every neural candidate. Algorithm 1 outlines how the morphing forecast, confidence interval, and probability estimation are integrated to produce the final score used for evolutionary selection.

Algorithm 1 Batch-Level Forecasting and Scoring in *ShouldYouEvenNN*

- 1: **Input:** validation accuracies $y_{1:t}$, batch indices $x_{1:t}$, baseline accuracy g
 - 2: **Compute horizon:** $T_f \leftarrow t_{\text{now}} + N \cdot \max(t_{\text{now}}, t_{\text{dataset}})$
 - 3: **Smooth** $y_{1:t}$ with exponential smoothing
 - 4: **Fit** rational f_r , sigmoid f_s , and linear f_l models to (x, y)
 - 5: **Estimate** local slope $s = \nabla y_t$ and curvature $c = \nabla^2 y_t$
 - 6: **Compute weights:** $(w_r, w_s, w_l) \leftarrow \text{softmax}_T(\phi(s, c))$
 - 7: **Morph forecast:** $f_{\text{morph}}(x) = w_r f_r(x) + w_s f_s(x) + w_l f_l(x)$
 - 8: **Predict** $\hat{a} = f_{\text{morph}}(T_f)$ and CI $[\hat{a}_{\text{low}}, \hat{a}_{\text{high}}]$
 - 9: **Estimate probabilities:** compute optimistic $m = \Pr[\mathcal{N}(\hat{a}_{\text{high}}, v) > g]$ and conservative $s = \sigma(((\hat{a}_{\text{high}} - g) - \pi(s, v))/\tau)$
 - 10: **Blend and stabilize:** $p_{\text{base}} \leftarrow \alpha m + (1 - \alpha)s$; $p \leftarrow \max\{0.05, 0.6p_{\text{base}} + 0.4y_t\}$
 - 11: **Output:** forecast $\hat{a}$, interval $[\hat{a}_{\text{low}}, \hat{a}_{\text{high}}]$, and probability p
-

Evolutionary Search and Selection

Following the forecasting stage, *ShouldYouEvenNN* enters a lightweight evolutionary loop that refines the population of neural candidates over successive generations. This process balances

exploration of new architectures with exploitation of those that already show promising early behavior. Each generation follows the steps below:

1. **Candidate spawning.** A mixed strategy creates new individuals: roughly half are sampled uniformly from the search space, while the remaining half are offspring produced by mutation and recombination of previously high-scoring architectures. All models are instantiated deterministically under a shared random context to guarantee reproducibility.
2. **Training phase.** Each candidate is trained for one epoch on its current instance budget. After training, the number of instances is expanded by a growth factor ($1.4\times$) for the next cycle, progressively increasing the amount of evidence without restarting from scratch.
3. **Forecasting and scoring.** The forecasting module projects each candidate’s validation accuracy to a future effort horizon, computes its confidence interval, and estimates the probability p that it will surpass the non-neural baseline. These probabilities are normalized into selection scores, providing a unified criterion for comparing architectures of different sizes and training histories.
4. **Hybrid pruning and selection.** The population is filtered using a multi-criterion strategy that integrates:
 - newborn protection for under-trained individuals,
 - convex lower-bound discard for clearly hopeless models,
 - stagnation detection for flat learning curves,
 - and probabilistic ranking with elite and incubator buffers to preserve both exploitative and exploratory diversity.

Survivors are ranked by their probability-weighted scores, and the pruning rate adapts dynamically to the generation count to balance convergence and diversity.

5. **Ledger update and continuation.** After pruning, all surviving candidates are logged with their architectures, metrics, and forecasts, forming the ledger for the next generation. Evolution continues until the global time budget is exhausted or no candidate achieves sufficient probability ($p > 0.5$) to justify further evaluation.

Decision and Refinement Phase

Expected-Utility Criterion

At the end of each generation, *ShouldYouEvenNN* evaluates the best-performing candidate through a Bayesian-inspired *expected-utility* formulation. This mechanism integrates the probability that a specific neural candidate will surpass the baseline, the expected magnitude of that improvement, and the remaining training cost required to confirm it.

Conceptual Definition. The scalar utility U expresses the trade-off between benefit and cost:

$$U = \mathbb{E}[\text{Benefit}] - \mathbb{E}[\text{Cost}],$$

where the first term represents the probability-weighted performance gain, and the second quantifies the normalized computational cost of reaching the forecast horizon.

Formal Components. Three quantities are combined:

- p : the probability that the top candidate will exceed the baseline validation accuracy, estimated from the hybrid Monte Carlo–sigmoid model.
- $\Delta = \hat{y}_f - g$: the expected validation-accuracy margin between the forecasted performance $\hat{y}_f$ and the baseline g .
- C : a normalized representation of the remaining training cost, computed from the projected horizon.

Relation to the Probability Estimate. The probability p computed during forecasting represents the model’s estimated likelihood of surpassing the non-neural baseline. However, this measure

alone does not account for the computational cost of continuing training. The expected-utility criterion extends this idea from a purely probabilistic judgment to a cost-aware decision rule. By combining the forecasted probability p , the expected improvement margin Δ , and the normalized remaining cost C , the framework assesses not only whether success is likely, but whether pursuing that success is rational under the current budget. In other words, p quantifies *belief*, whereas U quantifies *worth*.

The resulting expected utility is

$$U = p \cdot \Delta - \lambda C,$$

where λ regulates the penalty assigned to costly candidates.

Implementation in the Framework

In practice, the expected-utility computation follows a smoothed and compressed formulation implemented as:

$$U = w \cdot (p \cdot \Delta) - \text{cost_scale} \cdot \left(1 - e^{-\frac{E_{\text{future}} - E_{\text{spent}}}{c_s}}\right),$$

where:

- E_{future} and E_{spent} represent the total projected and current batch counts, respectively.
- c_s is a *cost-smoothing factor* (default 150) that compresses the cost growth over long horizons.
- $w = \text{evidence_min} + (1 - \text{evidence_min})(1 - e^{-E_{\text{spent}}/e_s})$ is an *evidence weight* that gradually increases confidence as more batches are processed, with e_s controlling the ramp steepness.

This formulation ensures that:

1. early in training, when evidence is scarce, w remains small—preventing premature optimism;
2. as training progresses, w approaches one, giving greater weight to reliable probability estimates;

3. the influence of excessively long forecast horizons is limited through exponential cost compression, preventing the utility from being dominated by distant projections.

The decision mechanism follows a two-stage logic:

1. If the expected utility $U > 0$, the framework interprets this as sufficient evidence that further neural training may be worthwhile. The top-ranked candidate is then selected for refinement under early stopping. If that candidate fails to surpass the baseline accuracy g , the next best candidates are refined sequentially until one succeeds or the remaining time budget is exhausted.
2. If no candidate achieves positive utility before the time budget expires, the framework still performs a final fallback refinement on the best forecasted model to confirm whether neural networks are indeed uncompetitive for the given dataset.

Triggering the Refinement Phase

If at any generation the expected utility becomes positive ($U > 0$), the search phase stops, and the corresponding candidate is selected for refinement. This refinement is performed under the remaining global time budget using a classical early-stopping mechanism with fixed patience and tolerance parameters. The model continues training until one of the following conditions is met:

1. Validation accuracy surpasses the baseline metric.
2. No measurable improvement occurs within the patience window.
3. The global time budget expires.

During this process, training and validation metrics are logged batch by batch, and the best-performing checkpoint is restored automatically.

Refinement Behavior. The refinement procedure allocates the remaining time budget between 30 seconds and the duration of the baseline MLP training, whichever is greater. The selected candidate continues learning under identical conditions to those used during the search phase,

ensuring comparability. The final model’s performance is summarized in terms of its validation accuracy, improvement margin, and total elapsed training time.

Final Decision Procedure

ShouldYouEvenNN produces two sequential decisions governing whether neural training is justified for a given dataset:

1. An **internal decision** within the evolutionary loop, based on the expected utility of the best candidate. If the expected utility becomes positive ($U > 0$), the framework interprets this as sufficient evidence that continued neural training is likely to be beneficial.
2. A **final decision** after refinement, verifying whether any refined candidate actually surpasses the non-neural baseline in validation accuracy.

Even when no candidate achieves positive expected utility before the time budget expires, the framework still performs a *fallback refinement* on the best forecasted architecture to empirically confirm whether neural models might still offer competitive performance. Thus, two refinement paths are possible:

- **Evidence-triggered refinement:** initiated when $U > 0$, targeting the top-ranked candidate and, if necessary, subsequent ones while time remains.
- **Fallback refinement:** executed when all $U \leq 0$, applying early stopping to the best forecasted model to validate that no neural advantage is being overlooked.

The framework updates its decision in both cases. If any refined model exceeds the baseline accuracy, the dataset is marked as *neural worthy*; otherwise, the conclusion is that neural networks are not competitive within the given computational budget.

Validation and Benchmarking

Outcome Logging and Reproducibility

All decisions, forecasts, and evaluation outcomes are stored in structured ledgers to ensure experiment reproducibility. Each ledger entry includes:

Algorithm 2 Overall Operational Flow of *ShouldYouEvenNN*.

Require: Training data $(X_{\text{train}}, y_{\text{train}})$, validation data $(X_{\text{val}}, y_{\text{val}})$, non-neural baseline accuracy g

Ensure: Final decision on whether neural training is worthwhile

- 1: **Initialize:** Load dataset, baselines and define search space parameters and population size.
 - 2: **Phase 1: Candidate Training**
 - 3: Train an initial population of neural architectures under limited batch budgets.
 - 4: Log validation accuracies y_t after each batch index t .
 - 5: **Phase 2: Forecasting**
 - 6: For each candidate, fit a morphing forecast model f_{morph} to predict future validation accuracy.
 - 7: Estimate probability of improvement p relative to baseline g .
 - 8: **Phase 3: Utility-Based Selection**
 - 9: Compute expected utility $U = p \cdot \Delta - \lambda C$ for the best candidate.
 - 10: **if** $U > 0$ **then**
 - 11: Trigger refinement phase (early-stopping training) on the top-ranked candidate.
 - 12: If this candidate fails to surpass baseline g , proceed sequentially with the next best candidates while time budget remains.
 - 13: **else**
 - 14: No candidate reached positive utility; perform fallback refinement on the best forecasted candidate.
 - 15: **end if**
 - 16: **Phase 4: Refinement and Validation**
 - 17: Continue training each selected candidate using early stopping until validation accuracy stabilizes or time expires.
 - 18: **if** any refined candidate surpasses baseline g **then**
 - 19: Mark dataset as *neural worthy* and record best validation accuracy.
 - 20: **else**
 - 21: Conclude that neural networks are not competitive under the given budget.
 - 22: **end if**
 - 23: **Return:** Final decision, best validation accuracy, best model, and total elapsed time.
-

- Candidate architecture parameters and random seed.
- Batch-level learning curve and forecasted accuracy.
- Confidence intervals and probability of surpassing the baseline.
- Utility values, cost components, and final refinement outcomes.

Deterministic random generators synchronize NumPy, PyTorch, and Python states, guaranteeing that identical seeds produce identical evolutionary trajectories and outcomes.

Forecast Strategy

To isolate the behavior of early-learning signals from the decision-making logic of *ShouldYouEvenNN*, a dedicated benchmark was executed. This experiment measured how reliably partial batch-level validation trajectories could approximate the final validation performance of neural architectures once fully trained, independent of pruning or evolutionary selection.

Setup. For each OpenML dataset, a population of $N = 25$ multilayer perceptron (MLP) architectures was randomly sampled from the same search space $\mathcal{S}$ described in Section Search Space. Training was executed on PyTorch with deterministic backends and seeded data loaders.

Every model was trained from scratch under a fixed batch-effort budget equivalent to three complete dataset passes or epochs. Each processed mini-batch incremented a deterministic counter, and validation accuracy was recorded after every batch, forming a sequence $\{(b_t, a_t)\}_{t=1}^T$ of cumulative effort b_t versus validation accuracy a_t . This batch-level resolution enabled consistent comparison across architectures with varying batch sizes.

Forecasting Procedure. For each recorded trajectory, a future batch horizon was defined as:

$$B_f = b_{\text{now}} + N \cdot \max(b_{\text{now}}, b_{\text{dataset}}),$$

where b_{dataset} is the number of batches per full dataset pass and $N = 50$ the extrapolation depth. Forecasts were generated by fitting curve-based models to the observed validation trajectory using the `forecast_with_ci` and `forecast_with_ci_morphing` functions in `forecaster.py`. These models (rational, sigmoid, and linear regressors) estimated a mean forecast $\hat{a}_f$ together with a two-tailed confidence interval $[\hat{a}_{\text{low}}, \hat{a}_{\text{high}}]$ derived from residual variance. When fewer than five validation points were available, the procedure defaulted to a conservative fallback $\hat{a}_f = a_T + 0.05$ to avoid overconfident projections.

Fidelity Verification. All architectures were subsequently retrained to convergence using early stopping. The realized validation accuracies a_{fidelity} were then compared to their predicted values $\hat{a}_f$, producing a per-dataset ledger of forecast fidelity. Evaluation metrics included the mean abso-

lute error (MAE), bias, and root-mean-square error (RMSE):

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{a}_{f,i} - a_{\text{fidelity},i}|, \quad \text{Bias} = \frac{1}{N} \sum_{i=1}^N (\hat{a}_{f,i} - a_{\text{fidelity},i}),$$

alongside Pearson and Spearman correlations quantifying linear and rank-order agreement between forecasted and final accuracies.

Visualization and Reporting. Each benchmark produced a structured ledger and summary statistics, together with three visual diagnostics:

1. **Batch-Level Learning Curves:** Validation accuracy over cumulative batches, including forecasted mean and confidence bounds.
2. **Forecast vs. Actual Scatter Plots:** Comparing predicted and realized accuracies for all candidates.
3. **Forecast Error Distributions:** Histograms of residuals ($a_{\text{fidelity}} - \hat{a}_f$) across datasets.

This evaluation seeks to confirm that early batch-level learning behavior contains sufficient predictive information to estimate the long-term accuracy of neural models with bounded uncertainty. By decoupling forecasting performance from the evolutionary process, the experiment validates that *ShouldYouEvenNN*’s forecasting mechanism is both stable and generalizable across datasets and architectures.

ShouldYouEvenNN Performance

To establish a consistent reference point, a non-neural benchmark was built using *NaiveAutoML* (Mohr & Wever, 2023), which evaluates a diverse portfolio of classical algorithms such as Gradient Boosting, Random Forests, and Logistic Regression. The hyperparameter optimization phase was deliberately omitted, to preserve the conceptual focus of the experiment: determining whether neural training should even proceed to that phase. By freezing the search before hyperparameter tuning, the benchmark represents a realistic and fixed non-neural performance ceiling—the level of accuracy that can be achieved with minimal computational investment. This ceiling serves as

the goal metric against which *ShouldYouEvenNN* estimates the probability that early-stage neural models could provide a meaningful improvement if further training or hyperparameter optimization were to be pursued.

In addition, two standard reference models were trained independently for each dataset: a Hist-GradientBoostingClassifier (HGB) and a standard sklearn Multilayer Perceptron (MLP). Their wall-clock training times and test accuracies were recorded to compare later against *ShouldYouEvenNN*. The effective time budget for *ShouldYouEvenNN* was determined by:

$$T_{\text{budget}}^{(\text{ShouldYouEvenNN})} = \max(T_{\text{HGB}} \cdot \beta_{\text{budget}}, T_{\text{MLP}}, 60),$$

where T_{HGB} denotes the training duration of the HGB baseline, β_{budget} is an adjustable multiplier controlling exploration depth (set to three in these experiments), and 60 seconds establishes a minimum allocation for datasets with small baseline costs.

Data Preparation. Each dataset was retrieved from OpenML and split into training, validation, and test partitions with an 80/10/10 ratio using stratified sampling. Categorical features were label-encoded, numerical features standardized, and missing values imputed using the mean or mode depending on data type. All features and targets were subsequently converted to PyTorch tensors.

Population and Search Space. For every dataset, a search space $\mathcal{S}$ of MLP neural architectures was instantiated based on the input and output dimensionalities of the data. The population size was fixed at $N = 25$, with candidates sampled uniformly from $\mathcal{S}$ (see Section Search Space).

Training Configuration. Each candidate began training with a limited instance budget corresponding to 10% of the available training data with the already described behavior in the increase. Validation and training metrics from these short runs were collected at batch level, serving as inputs for the forecasting and probabilistic scoring layers of *ShouldYouEvenNN*.

Evaluation Protocol. After the evolutionary phase concluded, the most promising candidate (if selected) was retrained until either (i) the validation accuracy surpassed the baseline metric or (ii) the time budget $T_{\text{budget}}^{(\text{ShouldYouEvenNN})}$ was exhausted. The final model was then evaluated on both validation and test sets. For each dataset, the validation and test accuracies, total time consumed,

probability of surpassing the baseline, and expected utility at the decision phase, were recorded.

Outputs and Logging. Each experiment produced detailed outputs including (i) per-generation population dynamics, and (ii) comprehensive summaries integrating baseline metrics, time budgets, evolutionary statistics, and final decisions on whether the neural model was worth training under the given constraints for later analysis.

External Benchmarking with LCBench

To contextualize the validation performance of the neural candidates discovered by *ShouldYouEvenNN*, an external benchmarking experiment was conducted using the **Learning Curve Benchmark (LCBench)** dataset (Zimmer et al., 2020). LCBench provides precomputed learning curves and final validation accuracies for a large set of neural architectures across tabular classification tasks, serving as a reliable upper reference for attainable neural performance.

For each dataset appearing in both the OpenML collection used in this study and LCBench, the final validation accuracy of the best *ShouldYouEvenNN* model was compared to the corresponding LCBench-reported accuracy. This analysis quantifies how closely the proposed lightweight, evolutionary framework can approximate the performance achieved by 200 fully trained networks per dataset as a benchmarking analysis. Descriptive statistics and a contingency breakdown of surpassing cases are reported in Section External Validation against LCBench.

Results



Early Signals Forecasting Strategy

To assess the reliability between the forecasting strategy and the "true" results for each model, the next results were analyzed.

In the Figure 1 and 3, two examples of the learning curves of some of the models trained according to the *ShouldYouEvenNN* strategy is observed, and that were later tested for two specific datasets (*ada* and *phoneme*). Figure 2 and 4 show the error distribution with its respective MAE.

The reliability of the forecasting mechanism was assessed across 32 datasets, summarizing its deviation and calibration with respect to the actual validation accuracies observed after full early-stopping training. As shown in Table , the mean absolute error (MAE) averaged 0.1465, indicating

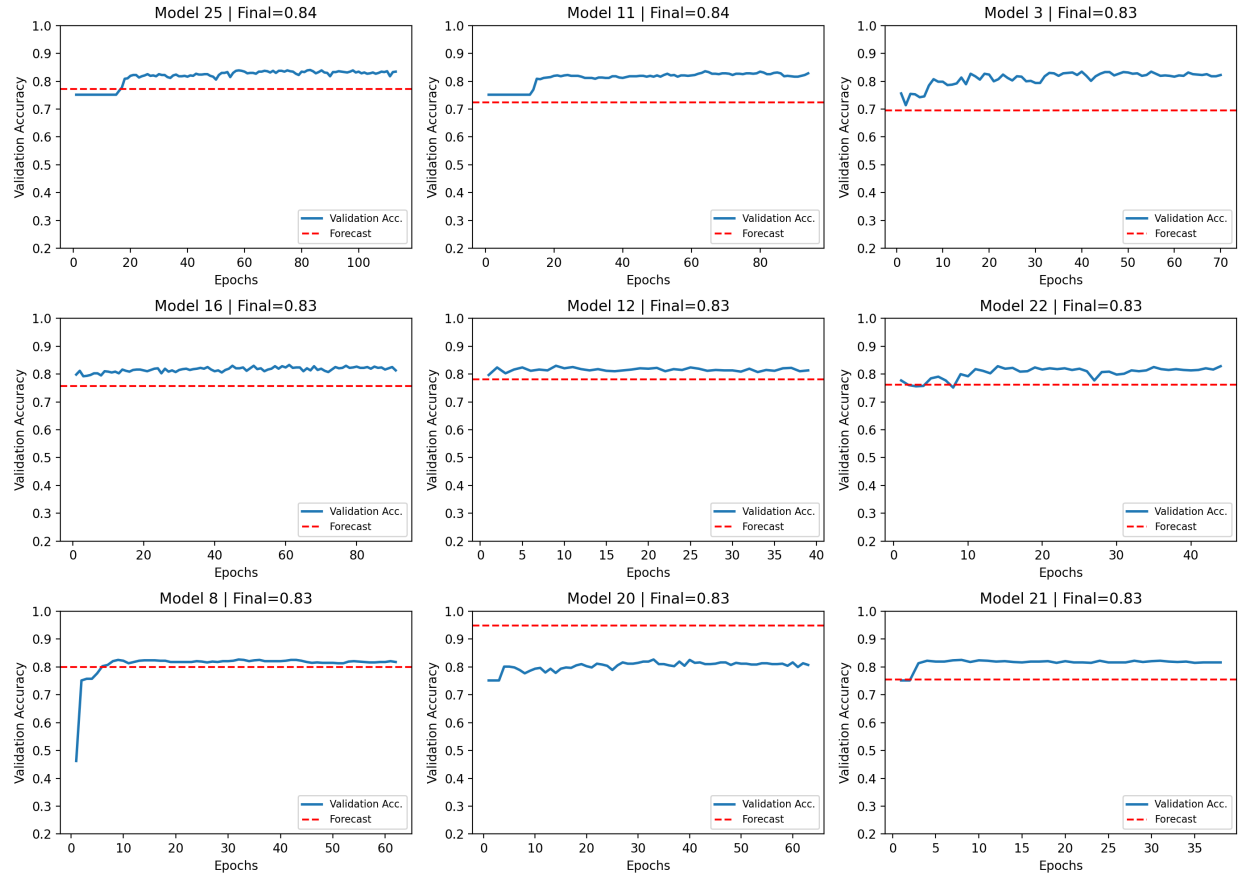
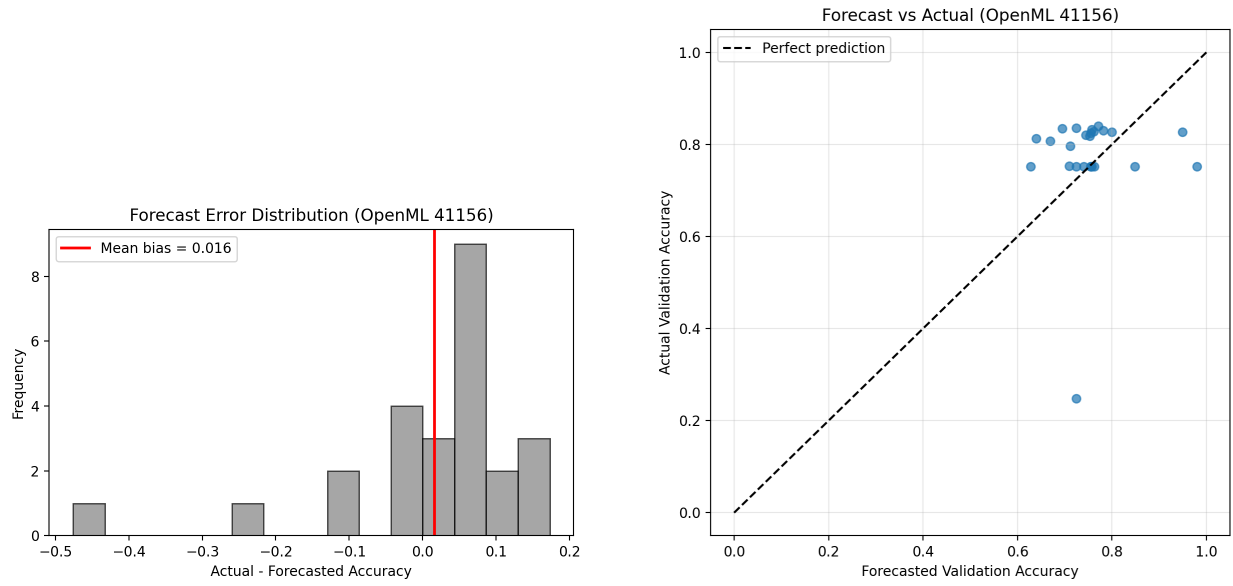


Figure 1

Example of learning curves for several models trained under the ShouldYouEvenNN strategy with their respective forecasted accuracy on the `ada` dataset. The trajectories illustrate the early-stage validation behavior used for forecasting.

that forecasts deviated by approximately fifteen percentage points from the true final performance on average. The corresponding root mean squared error (RMSE) of 0.1920 reflects the presence of occasional outliers in which the fitted curves diverged from the eventual validation trajectory, typically associated with noisy or stagnant learning curves.

A mild negative bias of -0.0324 suggests that forecasts tended to underestimate final performance as seen as well in Figure 2, a conservative behavior aligned with the framework’s goal of avoiding unnecessary neural investment. The average confidence interval (CI) coverage was 0.56, below the nominal 95% expectation, revealing that the uncertainty estimates were somewhat overconfident. This is consistent with the observed mean CI width of 0.26, which proved too narrow relative to



(a) Error distribution of forecasted validation accuracies with corresponding mean absolute error (MAE).

(b) Scatter plot comparing forecasted and observed validation accuracies.

Figure 2

Forecasting performance visualization on the *ada* dataset.

the empirical forecast error. Nevertheless, this conservative calibration minimizes false positives during the early rejection phase.

The correlation coefficients between forecasted and actual accuracies were modest ($r = 0.20$, $\rho = 0.21$), implying that while the forecasts captured the general direction of performance evolution, they were not strongly predictive of candidate rankings. This behavior highlights that the forecasting subsystem functions effectively as a binary decision aid—indicating whether further training is justified—rather than as a precise performance predictor.

ShouldYouEvenNN Performance

The evolution observed in Figure 5 exemplifies the internal behavior of the framework during the search phase. The mean validation accuracy progressively increases across generations, accompanied by closely aligned forecasted accuracies, indicating that the forecasting module provides a realistic and conservative estimate of performance trends. The probability of surpassing the baseline (p) rises steadily during the first half of the process, reflecting growing evidence that

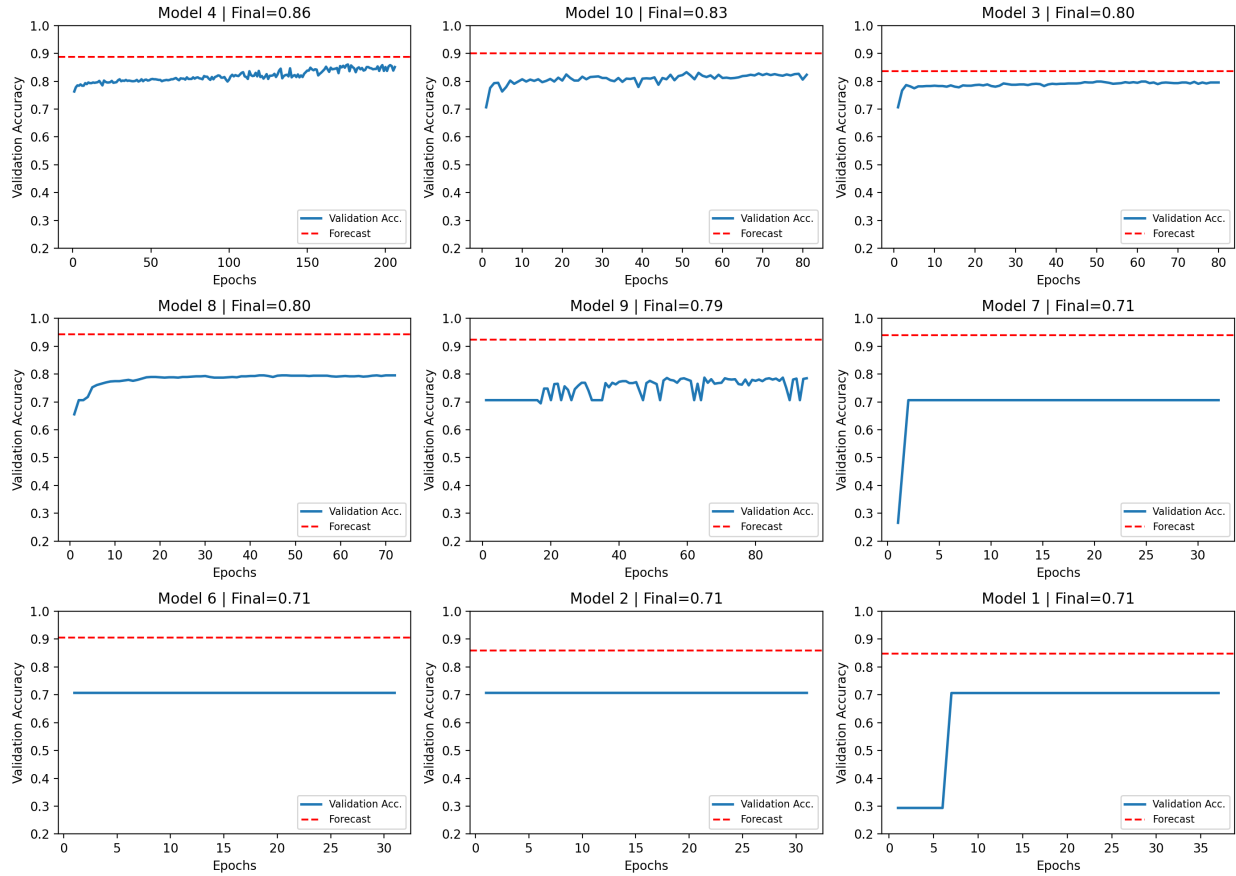


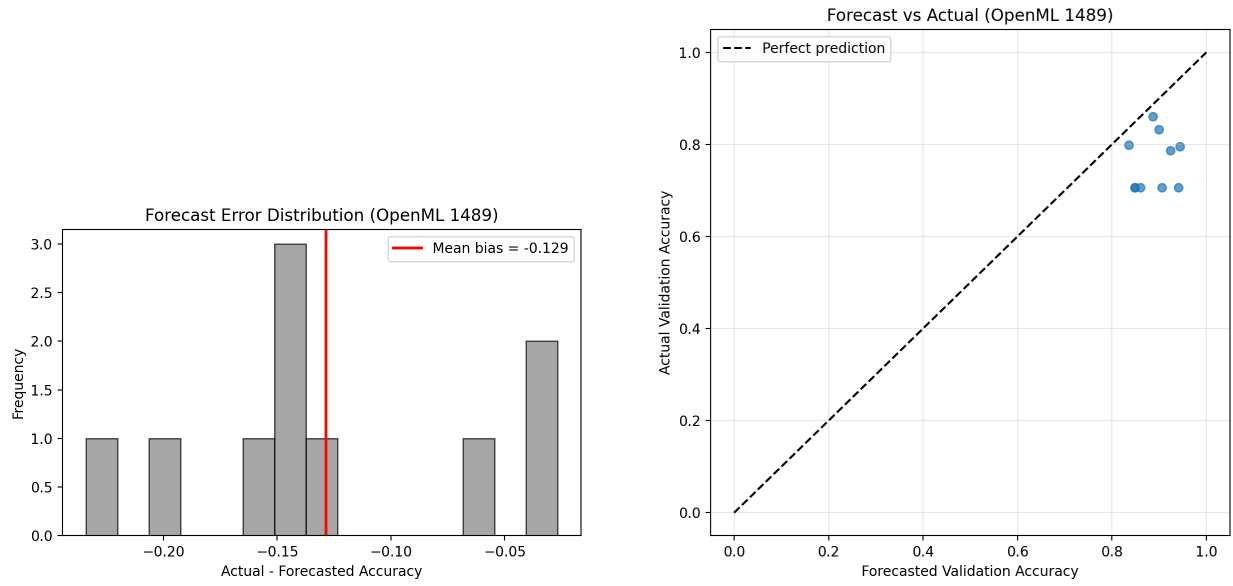
Figure 3

Example of learning curves for several models with their respective forecasted accuracy on the phoneme dataset.

the population contains promising architectures. However, the expected utility (U) remains near zero or slightly negative, showing that the projected computational cost continues to outweigh the marginal expected gain—hence no refinement is triggered at this stage. The cumulative compute curve increases almost linearly, confirming that the training effort per generation is stable and well-bounded by the global budget. Overall, this plot illustrates a typical non-triggering scenario where learning signals improve and forecasts remain well calibrated, yet the expected-utility criterion correctly prevents unnecessary extended training.

After validating the forecasting module, the full algorithm was executed across all benchmark datasets. The aggregated outcomes are summarized in Table 3.

Across the 71 OpenML datasets evaluated, the ShouldYouEvenNN framework identified neural



(a) Error distribution of forecasted validation accuracies with corresponding mean absolute error (MAE).

(b) Scatter plot comparing forecasted and observed validation accuracies.

Figure 4

Forecasting performance visualization on the *phoneme* dataset.

networks as *worth training* in 38 cases, representing just over half of the total sample. Among these, 16 datasets achieved validation accuracies that surpassed their respective non-neural baselines, and 8 maintained this advantage on the held-out test sets.

These results indicate that while *ShouldYouEvenNN*'s decision rule is selective, it remains capable of identifying scenarios in which neural networks provide tangible benefits over strong ensemble-based baselines. The gap between the number of neural-worthy cases and those that ultimately surpassed validation and test metrics illustrates the framework's conservative stance: only a subset of early promising signals translated into verifiable generalization gains. This outcome aligns with *ShouldYouEvenNN*'s primary objective—to allocate additional training effort only when statistically justified by early evidence—thereby reducing wasted computation without systematically overlooking genuinely competitive neural configurations.

Figure 6 illustrates how the expected utility assigned by the decision rule varies with dataset feature dimensionality (in logarithmic scale). Each point represents a dataset, where orange markers cor-

Table 2

Summary statistics of the EBE-NAS results.

	mean	std
MAE	0.1465	0.0700
Bias	-0.0324	0.0948
RMSE	0.1920	0.0860
CI coverage	0.5557	0.3201
Avg CI width	0.2574	0.1271
Pearson r	0.2035	0.3270
Spearman rho	0.2149	0.2863

respond to cases deemed *neural worthy* (decision = True) and blue markers to those rejected. Cross-shaped markers further indicate instances in which the validation baseline was actually surpassed after extended training.

The plot reveals a consistent separation between positive and negative utility regions. Most datasets with positive expected utility ($U > 0$) cluster below a few hundred features, suggesting that ShouldYouEvenNN tends to favor neural architectures when the feature space is relatively low-dimensional and structured. Conversely, for high-dimensional datasets (beyond 10^3 features), expected utility values remain largely negative, implying that the computational cost or instability outweighs potential gains in these scenarios.

Interestingly, several datasets near the decision boundary ($U \approx 0$) correspond to borderline cases where the neural models approached but did not exceed baseline performance. This reinforces the probabilistic nature of the decision function: small variations in early learning dynamics can tip the forecast from rejection to acceptance. The figure thus encapsulates the framework’s core behavior—balancing optimism and cost sensitivity—while highlighting that the conditions under which neural networks become worthwhile remain concentrated in medium-sized, moderately complex datasets rather than in very high-dimensional feature spaces. Also, the cases where the decision was defined to be true and $U < 0$ correspond to scenarios where the framework initially failed to identify a task as neural worthy, but after extending the best candidate during the refinement phase, the model ultimately surpassed the baseline metric. The percentage of these cases is very

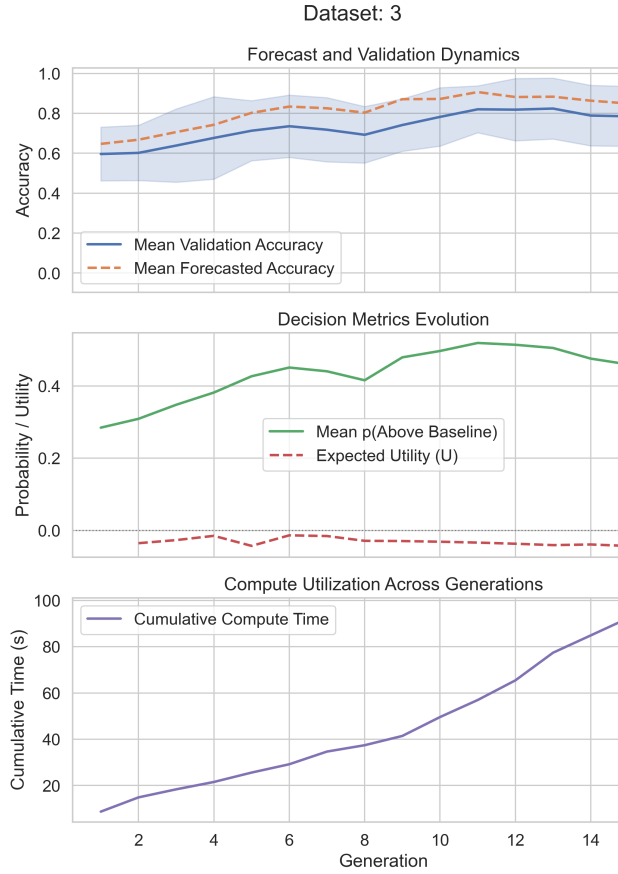


Figure 5

*Generation-level dynamics for Dataset 3. The figure summarizes how validation accuracy, forecasted performance, probability of surpassing the baseline, and expected utility evolved through successive generations of the *ShouldYouEvenNN* framework.*

low, which shows the robustness and conservative reliability of the decision mechanism. In other words, false negatives—datasets where *ShouldYouEvenNN* prematurely rejected neural exploration despite eventual success—are rare, but are able to be identified by the tool with this last failguard. This confirms that the expected-utility criterion favors caution, prioritizing the prevention of unnecessary training over speculative optimism.

External Validation against LCBench

To assess whether the architectures identified by *ShouldYouEvenNN* approach the quality of fully trained neural networks reported in standardized NAS benchmarks, we compared their final validation accuracies to those of the **Learning Curve Benchmark (LCBench)** (Zimmer et al., 2020).

Metric	Value
Total Datasets	71
Neural Worthy	38
Validation Surpassed	16
Test Surpassed	8

Table 3

Overall results of the ShouldYouEvenNN evaluation across all datasets.

LCBench includes performance records for hundreds of neural configurations per dataset, providing an empirical ceiling for expected neural performance.

Table 4 summarizes the aggregated statistics of the comparison across 35 overlapping datasets.

Table 4

Comparison of final validation accuracies between ShouldYouEvenNN and LCBench.

Metric	Mean	Std	Min	25%	50%	75%	Max
EBE Final Val. Acc.	0.771	0.231	0.006	0.712	0.819	0.926	0.994
LCBench Final Val. Acc.	0.819	0.155	0.290	0.743	0.838	0.945	0.997
Difference (EBE – LCBench)	-0.048	0.159	-0.871	-0.037	-0.004	0.008	0.097

Across datasets, the average deviation between the best *ShouldYouEvenNN* model and its LCBench counterpart was approximately -0.05 , with standard deviation 0.16 . This indicates that the proposed framework, despite its strict time and data constraints, achieves validation accuracies within five percentage points of the high-fidelity LCBench results on average.

A contingency analysis of surpassing outcomes is shown in Table 5.

Table 5

Cross-tabulation of datasets where ShouldYouEvenNN and LCBench surpass their respective baselines.

LCBench surpassed	ShouldYouEvenNN surpassed baseline	Count
False	False	21
True	False	8
True	True	5
False	True	1

Overall, in roughly 37% of the overlapping datasets (13/35), the final models obtained by *ShouldY-*

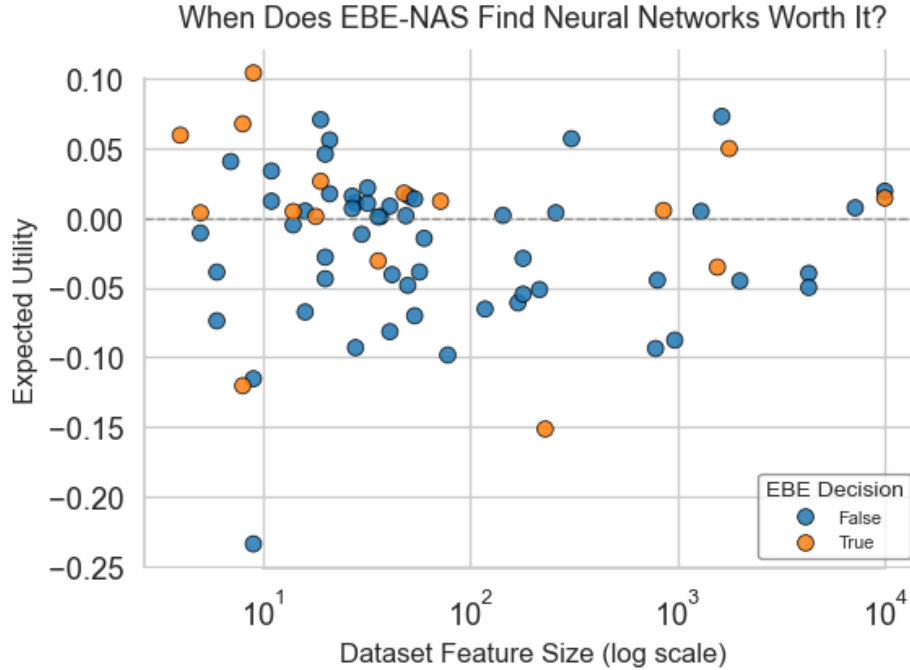


Figure 6
Dataset Feature Size vs U

ouEvenNN matched or exceeded the validation accuracy reported by LCBench. The remaining discrepancies are modest and primarily reflect the framework’s intentionally conservative stopping policy. These results validate that the early-forecast-driven selection mechanism can recover high-performing neural architectures that remain competitive with exhaustive NAS baselines while preserving computational efficiency.

Discussion

Interpretation and Theoretical Positioning

The results of *ShouldYouEvenNN* validate its central hypothesis: that early training dynamics contain sufficient predictive information to make rational, cost-aware decisions about whether neural networks are worth pursuing. Across the OpenML benchmark, the framework consistently identified competitive architectures without relying on exhaustive convergence, confirming that lightweight forecasting and selective continuation can substitute for full training in many cases. These findings directly address the long-standing imbalance between AutoML and NAS efficiency. As noted by Baratchi et al. (2024) and Mohr and Wever (2023), AutoML systems achieve

scalable performance through separable and interpretable optimization, whereas classical NAS pipelines—often based on reinforcement or evolutionary search (Mendoza et al., 2016; Zoph & Le, 2016)—still assume that all candidates warrant full convergence. *ShouldYouEvenNN* challenges this assumption by introducing an explicit decision layer that governs when neural exploration itself is justified, aligning with the resource-rational principles discussed by Mohr and van Rijn (2022). This integration of decision theory with early-signal evidence redefines NAS as a process of *judgment under uncertainty* rather than blind optimization.

Forecasting Reliability and Early-Signal Evidence

The batch-level forecasting component proved both robust and informative. Its morphing curve model, which blends rational, sigmoid, and linear regressors, captured meaningful learning trends even under noisy, low-data conditions. This empirically supports prior claims that partial learning curves are predictive of final outcomes (Domhan et al., n.d.; White et al., 2021; Wistuba & Pedapati, 2020). However, unlike those studies—where forecasts serve as passive ranking tools—*ShouldYouEvenNN* operationalizes them as actionable evidence within a decision pipeline.

The conservative bias observed in the fidelity benchmark is consistent with the cautionary calibration suggested by Egele et al. (2024), who showed that one-epoch signals can be effective predictors when optimism is moderated. By penalizing steep or volatile learning curves, the forecasting mechanism minimizes false positives, favoring stability and reproducibility. This confirms that early-stage training signals can function as probabilistic priors guiding resource allocation, not merely as performance predictors.

Decision Criteria, Conservativeness, and Efficiency

The expected-utility formulation

$$U = p \cdot \Delta - \lambda C$$

proved to be a reliable control mechanism for balancing potential gains against training cost. By requiring $U > 0$ before triggering refinement, the framework prioritizes statistical confidence over speculative exploration. This design choice explains the relatively small proportion of datasets

marked as ‘neural worthy’ but also the rarity of false positives during refinement. In practice, *ShouldYouEvenNN* acts as an *efficiency filter* rather than a discovery engine.

The hybrid pruning logic—combining convex lower-bound discard, stagnation detection, and probabilistic ranking—builds upon multi-fidelity and early-stopping strategies (Garcia-Garcia et al., 2025; Jurado & Vanschoren, 2022; Ren et al., 2022; Sakamoto et al., 2023), but extends them with a principled probabilistic rule for whether further neural training is justified. This elevates cost-awareness from a heuristic to a formal, interpretable criterion.

Computationally, the framework achieves near-baseline runtimes while evaluating diverse populations, confirming that batch-based effort representation enables fine-grained budget control. This efficiency directly aligns with Mohr and Wever (2023), who emphasized that simplicity and determinism can outperform overly complex optimization pipelines.

Comparison with Baselines and Literature

The selective activation of the “neural worthy” decision primarily on medium-sized and moderately complex datasets mirrors the asymmetry reported by Shwartz-Ziv and Armon (2022), Borisov et al. (2022), and Grinsztajn et al. (2022). Deep models rarely dominate in tabular domains unless nonlinear feature interactions are strong or dataset scale is large, as also observed by McElfresh et al. (2023). *ShouldYouEvenNN* quantifies this boundary empirically, identifying the point at which neural exploration becomes rational given cost constraints.

External comparison with the LCBench benchmark (Zimmer et al., 2020) further positions the framework within the NAS landscape. Despite its lightweight evaluation regime, *ShouldYouEvenNN* achieved validation accuracy  within approximately five percentage points of LCBench results on average, and matched or exceeded them in over one-third of shared datasets. This proximity demonstrates that early-evidence forecasting can recover high-quality architectures typically found through exhaustive search, supporting the general efficiency conclusions of Egele et al. (2023) and Wei et al. (2020).

Practical Implications and Future Work

Beyond its methodological contributions, *ShouldYouEvenNN* offers practical value for automated pipelines that must decide whether neural training is justified at all. By turning early evidence into a decision signal, it reduces energy consumption and computational waste without systematically discarding competitive candidates. Moreover, the analysis of datasets that consistently trigger positive expected utility could inform meta-learning models capable of predicting ‘neural worthiness’ a priori.

Future research should explore adaptive calibration of the expected-utility parameters (e.g., cost scale and evidence ramp) based on dataset meta-features, and extend the framework to other architectures such as CNNs or Transformers. Integrating such decision layers into end-to-end AutoML systems would advance the field toward a fully autonomous modeling paradigm—one that not only trains models, but also determines, with statistical rigor, when training is worth pursuing.

Conclusion

The *ShouldYouEvenNN* framework demonstrates that early-training information, when properly modeled and integrated into a probabilistic decision process, can substantially reduce the computational burden of neural architecture search without compromising predictive reliability. By treating the continuation of neural training as a decision under uncertainty rather than a fixed commitment, the framework introduces a principled mechanism for determining when neural exploration is justified relative to strong non-neural baselines.

This study redefines the purpose of NAS. Rather than framing architecture discovery as a competition for the most accurate network, *ShouldYouEvenNN* interprets it as a problem of rational evidence allocation—where limited computational resources are invested only when early learning signals indicate a positive expected return. In this sense, NAS becomes not merely an optimization process, but an adaptive inference system guided by the probabilistic value of continuing to learn. Empirical results confirm that this perspective is both practical and generalizable. The framework consistently identified when neural networks were likely to be competitive, producing selections that closely approached high performing models while maintaining cost efficiency. Forecasting

bias remained intentionally conservative, ensuring that decisions favored reliability over speculative optimism. These findings highlight that early-training signals can serve not only as predictors of performance but as the statistical foundation for rational, data-driven control over the learning process itself.

Beyond its immediate results, this work contributes conceptually to the broader field of automated model design. It bridges the efficiency focus of AutoML with the representational depth of NAS, demonstrating that automation can evolve from search to judgment—from exhaustive exploration toward selective, evidence-based continuation. By unifying forecasting, probabilistic reasoning, and evolutionary refinement within a single decision framework, *ShouldYouEvenNN* establishes a reproducible path toward fast, transparent, and resource-aware neural model discovery.

Future extensions of this research could explore adaptive calibration of the expected-utility parameters, hierarchical decision layers across modalities (e.g., vision or time series), and tighter integration with pipeline-level AutoML systems. Such developments would move the field closer to a fully autonomous modeling paradigm—one that not only automates training, but also decides, with statistical rigor, when training is worth pursuing at all.

Referencias

- Aach, M., Inanc, E., Sarma, R., Riedel, M., & Lintermann, A. (2023). Optimal Resource Allocation for Early Stopping-based Neural Architecture Search Methods.
- Adriaensen, S., Rakotoarison, H., Müller, S., & Hutter, F. (2023). Efficient Bayesian Learning Curve Extrapolation using Prior-Data Fitted Networks. <https://arxiv.org/pdf/2310.20447>
- Baratchi, M., Wang, C., Limmer, S., van Rijn, J. N., Hoos, H., Bäck, T., & Olhofer, M. (2024). Automated machine learning: past, present and future. *Artificial Intelligence Review*, 57(5), 122. <https://doi.org/10.1007/s10462-024-10726-1>
- Borisov, V., Leemann, T., Seßler, K., Haug, J., Pawelczyk, M., & Kasneci, G. (2022). Deep Neural Networks and Tabular Data: A Survey. *IEEE Transactions on Neural Networks and Learning Systems*, 35(6), 7499–7519. <https://doi.org/10.1109/TNNLS.2022.3229161>
- Domhan, T., Springenberg, T., & Hutter, F. (n.d.). Speeding up Automatic Hyperparameter Optimization of Deep Neural Networks by Extrapolation of Learning Curves.
- Egele, R., Guyon, I., Sun, Y., & Balaprakash, P. (2023). Is One Epoch All You Need For Multi-Fidelity Hyperparameter Optimization? *ESANN 2023 proceedings*, 555–560. <https://doi.org/10.14428/esann/2023.ES2023-84>
- Egele, R., Mohr, F., Viering, T., & Balaprakash, P. (2024). The unreasonable effectiveness of early discarding after one epoch in neural network hyperparameter optimization. *Neurocomputing*, 597, 127964. <https://doi.org/10.1016/J.NEUCOM.2024.127964>
- Elsken, T., Metzen, J. H., & Hutter, F. (2019). *Neural Architecture Search: A Survey* (tech. rep.). <http://jmlr.org/papers/v20/18-598.html>.
- Garcia-Garcia, C., Derbel, B., Morales-Reyes, A., & Escalante, H. (2025). *Speeding up the Multi-objective NAS Through Incremental Learning* (Vol. 15247 LNAI). https://doi.org/10.1007/978-3-031-75543-9_{_}1
- Gardner, J., Popović, Z., Schmidt, L., & Allen, P. G. (2022). Subgroup Robustness Grows On Trees: An Empirical Baseline Investigation. <https://arxiv.org/pdf/2211.12703>

- Gorishniy, Y., Rubachev, I., Khrulkov, V., & Babenko, A. (2021). Revisiting Deep Learning Models for Tabular Data. *Advances in Neural Information Processing Systems*, 23, 18932–18943. <https://arxiv.org/pdf/2106.11959>
- Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on tabular data? <https://arxiv.org/pdf/2207.08815>
- Guan, J., Zhang, F., Liu, J., Du, X., Shen, X., & Shen, X. A. (2025). A Systematic Study on Early Stopping Metrics in HPO and the Implications of Uncertainty. *18*(6), 2150–8097. <https://doi.org/10.14778/3725688.3725689>
- Jin, H., Song, Q., & Hu, X. (2019). Auto-Keras: An Efficient Neural Architecture Search System. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1946–1956. <https://doi.org/10.1145/3292500.3330648>
- Jurado, I. C., & Vanschoren, J. (2022). Multi-fidelity optimization method with asynchronous generalized island model for AutoML. *GECCO 2022 Companion - Proceedings of the 2022 Genetic and Evolutionary Computation Conference*, 220–223. <https://doi.org/10.1145/3520304.3528917>
- Lin, Y., Endo, Y., Lee, J., & Kamijo, S. (2024). Bandit-NAS: Bandit sampling and training method for Neural Architecture Search. *Neurocomputing*, 597, 127684. <https://doi.org/10.1016/J.NEUCOM.2024.127684>
- Liu, X., Tian, J., Duan, P., Yu, Q., Wang, G., & Wang, Y. (2024). GrMoNAS: A granularity-based multi-objective NAS framework for efficient medical diagnosis. *Computers in Biology and Medicine*, 171, 108118. <https://doi.org/10.1016/J.COMPBIOMED.2024.108118>
- Lu, S., Hu, Y., Yang, L., Mei, J., Sun, Z., Tan, J., & Song, C. (2024). PHD-NAS: Preserving helpful data to promote Neural Architecture Search. *Neurocomputing*, 587. <https://doi.org/10.1016/j.neucom.2024.127646>
- McElfresh, D., Khandagale, S., Valverde, J., Prasad, V. C., Ramakrishnan, G., Goldblum, M., & White, C. (2023). When Do Neural Nets Outperform Boosted Trees on Tabular Data? *Advances in Neural Information Processing Systems*, 36. <https://arxiv.org/pdf/2305.02997>

- Mebirouk, N., Amrane, M., Messast, S., & Mazouzi, S. (2024). Enhanced analysis of landslide susceptibility mapping in the proximity of main roads in the province of Skikda, Algeria: using NAS for efficient performance and faster processing. *Modeling Earth Systems and Environment*, 10(5), 6449–6474. <https://doi.org/10.1007/s40808-024-02129-6>
- Mendoza, H., Klein, A., Feurer, M., Springenberg, J. T., & Hutter, F. (2016, November). Towards Automatically-Tuned Neural Networks. In F. Hutter, L. Kotthoff, & J. Vanschoren (Eds.), *Proceedings of the workshop on automatic machine learning* (pp. 58–65, Vol. 64). PMLR. https://proceedings.mlr.press/v64/mendoza_towards_2016.html
- Mohr, F., & van Rijn, J. N. (2022). Learning Curves for Decision Making in Supervised Machine Learning – A Survey. <http://arxiv.org/abs/2201.12150>
- Mohr, F., & van Rijn, J. N. (2023). Fast and Informative Model Selection Using Learning Curve Cross-Validation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(8), 9669–9680. <https://doi.org/10.1109/TPAMI.2023.3251957>
- Mohr, F., & Wever, M. (2023). Naive automated machine learning. *Machine Learning*, 112(4), 1131–1170. <https://doi.org/10.1007/s10994-022-06200-0>
- Nandal, P., Mann, P., Bohra, N., Aldehim, G., Elnour, A. A. H., & Allafi, R. (2025). Tropical cyclone intensity estimation based on YOLO-NAS using satellite images in real time. *Alexandria Engineering Journal*, 113, 227–241. <https://doi.org/10.1016/j.aej.2024.10.072>
- Nguyen, D. A., Kononova, A. V., Menzel, S., Sendhoff, B., & Back, T. (2022). An Efficient Con-
testing Procedure for AutoML Optimization. *IEEE Access*, 10, 75754–75771. <https://doi.org/10.1109/ACCESS.2022.3192036>
- Patel, S., Patel, D., Patel, A., Nanavati, J., Patel, U., Faldu, A., & Shingala, A. (2024). Accurate and Efficient Real-Time Crop and Weed Identification Using YOLO-NAS: A Neural Architecture Search-Optimized Object Detection Model. *International Journal of Engineering Trends and Technology*, 72(10), 35–45. <https://doi.org/10.14445/22315381/IJETT-V72I10P104>

- Ren, P., Xiao, Y., Chang, X., Huang, P.-y., Li, Z., Chen, X., & Wang, X. (2022). A Comprehensive Survey of Neural Architecture Search. *ACM Computing Surveys*, 54(4), 1–34. <https://doi.org/10.1145/3447582>
- Ru, B., Lyle, C., Schut, L., Fil, M., Van Der Wilk, M., & Gal, Y. (n.d.). Speedy Performance Estimation for Neural Architecture Search. <https://github.com/rubinxin/TSE>.
- Sakamoto, K., Ishibashi, H., Sato, R., Shirakawa, S., Akimoto, Y., & Hino, H. (2023). ATNAS: Automatic Termination for Neural Architecture Search. *Neural Networks*, 166, 446–458. <https://doi.org/10.1016/J.NEUNET.2023.07.011>
- Salehin, I., Islam, M. S., Saha, P., Noman, S., Tuni, A., Hasan, M. M., & Baten, M. A. (2024). AutoML: A systematic review on automated machine learning with neural architecture search. *Journal of Information and Intelligence*, 2(1), 52–81. <https://doi.org/10.1016/j.jiixd.2023.10.002>
- Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84–90. <https://doi.org/10.1016/J.INFFUS.2021.11.011>
- Solís-Martín, D., Galán-Páez, J., & Borrego-Díaz, J. (2025). A Model for Learning-Curve Estimation in Efficient Neural Architecture Search and Its Application in Predictive Health Maintenance. *Mathematics 2025, Vol. 13, Page 555, 13(4)*, 555. <https://doi.org/10.3390/MATH13040555>
- Swersky, K., Snoek, J., & Adams, R. P. (2014). Freeze-Thaw Bayesian Optimization. <https://arxiv.org/pdf/1406.3896>
- Thornton, C., Hutter, F., Hoos, H. H., & Leyton-Brown, K. (2013). Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. <https://arxiv.org/abs/1208.3719>
- van Hoof, J., & Vanschoren, J. (2021). Hyperboost: Hyperparameter Optimization by Gradient Boosting surrogate models. <https://arxiv.org/pdf/2101.02289>

- Wang, W., & Li, H. (2025). NAS FD Lung: A novel lung assist diagnostic system based on neural architecture search. *Biomedical Signal Processing and Control*, 100. <https://doi.org/10.1016/j.bspc.2024.107022>
- Wang, Z., Gehring, C., Kohli, P., & Jegelka, S. (2017). Ensemble Bayesian Optimization. *International Conference on Artificial Intelligence and Statistics, AISTATS 2018*, 745–754. <https://arxiv.org/pdf/1706.01445>
- Wei, C., Niu, C., Tang, Y., Wang, Y., Hu, H., & Liang, J. (2020). NPENAS: Neural Predictor Guided Evolution for Neural Architecture Search. *IEEE Transactions on Neural Networks and Learning Systems*, 34(11), 8441–8455. <https://doi.org/10.1109/TNNLS.2022.3151160>
- Wen, W., Liu, H., Chen, Y., Li, H., Bender, G., & Kindermans, P. J. (2019). Neural Predictor for Neural Architecture Search. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 12374 LNCS, 660–676. https://doi.org/10.1007/978-3-030-58526-6_{_}39
- White, C., Zela, A., Ru, B., Liu, Y., & Hutter, F. (2021). How Powerful are Performance Predictors in Neural Architecture Search? *Advances in Neural Information Processing Systems*, 34, 28454–28469. <https://arxiv.org/pdf/2104.01177>
- Wistuba, M., & Pedapati, T. (2020). Learning to Rank Learning Curves.
- Wu, J., Cao, M., Shan, L., & Yang, Q. (n.d.). Higher Performance for AutoML: The Benefit of Various Ensemble Bayesian Optimization Strategy. https://github.com/SupUnicorn/bbo_challenge_4th.
- Yan, S., White, C., Savani, Y., & Hutter, F. (2021). NAS-Bench-x11 and the Power of Learning Curves. *Advances in Neural Information Processing Systems*, 27, 22534–22549. <https://arxiv.org/pdf/2111.03602>
- Zheng, X., Ji, R., Chen, Y., Wang, Q., Zhang, B., Chen, J., Ye, Q., Huang, F., & Tian, Y. (2021). MIGO-NAS: Towards Fast and Generalizable Neural Architecture Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(9), 2936–2952. <https://doi.org/10.1109/TPAMI.2021.3065138>

- Zimmer, L., Lindauer, M., & Hutter, F. (2020). Auto-PyTorch Tabular: Multi-Fidelity MetaLearning for Efficient and Robust AutoDL. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(9), 3079–3090. <https://doi.org/10.1109/TPAMI.2021.3067763>
- Zoph, B., & Le, Q. V. (2016). Neural Architecture Search with Reinforcement Learning. *5th International Conference on Learning Representations, ICLR 2017 - Conference Track Proceedings*. <https://arxiv.org/abs/1611.01578v2>

Appendix

Appendix A: Learning Curves vs Forecast

Table 6

Condensed EBE-NAS Experiment Results per Dataset

data_id	naml_best_validation_metric	naml_test_accuracy	ebe_decision	ebe_val_acc	ebe_test_acc	ebe_surpassed_val_baseline	ebe_surpassed_test_baseline	ebe_time_used_s
3	0.9941	0.9969	False	0.9941	0.9891	True	False	61.3809
12	0.9850	0.9725	False	0.9750	0.9550	False	False	61.4269
23	0.5619	0.5153	True	0.6017	0.4881	True	False	9.0699
31	0.7575	0.7300	True	0.7562	0.7100	False	False	19.7634
54	0.8515	0.8412	True	0.8824	0.7529	True	False	9.3394
181	0.6050	0.6364	True	0.6345	0.5690	True	False	6.6603
188	0.6661	0.6622	True	0.6780	0.6554	True	False	5.2224
1049	0.9179	0.8836	True	0.9060	0.9110	False	True	19.5543
1067	0.8870	0.8412	True	0.8787	0.8318	False	False	27.6183
1111	0.9822	0.9822	False	0.9822	0.9822	True	True	86.5667
1169	0.6579	0.6607	True	0.6256	0.6293	False	False	93.6522
1457	0.8133	0.8600	True	0.5708	0.4933	False	False	177.8856
1461	0.9133	0.9060	True	0.8831	0.8830	False	False	80.0106
1464	0.7817	0.7733	True	0.8083	0.7867	True	True	3.7188
1468	0.9353	0.9398	True	0.9769	0.9167	True	False	7.0944
1475	0.6047	0.6258	True	0.5444	0.5368	False	False	68.5845
1486	0.9699	0.9697	False	0.9476	0.9504	False	False	90.4251
1487	0.9491	0.9310	True	0.9581	0.9231	True	False	19.1388
1489	0.9210	0.8918	False	0.8960	0.8788	False	False	73.6999
1494	0.8911	0.8720	True	0.8876	0.8768	False	True	20.5712

Continued on next page

Table 6
Condensed EBE-NAS Experiment Results per Dataset

data_id	naml_best_validation_metric	naml_test_accuracy	ebe_decision	ebe_val_acc	ebe_test_acc	ebe_surpassed_val_baseline	ebe_surpassed_test_baseline	ebe_time_used_s
1515	0.9087	0.9130	True	0.8261	0.7652	False	False	25.4087
1590	0.8734	0.8774	False	0.8237	0.8310	False	False	90.7140
1596	0.9507	0.9551	False	0.7219	0.7233	False	False	237.2460
4134	0.7939	0.8362	True	0.8050	0.7963	True	False	36.5215
4135	0.9502	0.9469	False	0.9420	0.9422	False	False	73.4512
4534	0.9755	0.9724	False	0.9700	0.9679	False	False	71.5146
4538	0.6709	0.6911	True	0.5652	0.5727	False	False	69.8245
4541	0.6131	0.6152	True	0.5851	0.5825	False	False	90.6034
23512	0.7197	0.7260	True	0.7161	0.7178	False	False	90.3425
23517	0.5233	0.5205	True	0.5059	0.5061	False	False	90.4725
40498	0.6718	0.6827	True	0.5778	0.6153	False	False	62.7789
40668	0.8272	0.8260	False	0.6609	0.6563	False	False	90.3981
40670	0.9582	0.9483	False	0.9529	0.9310	False	False	62.0052
40685	0.9998	0.9995	False	0.7860	0.7859	False	False	88.2175
40701	0.9604	0.9630	False	0.8588	0.8590	False	False	61.7084
40900	0.9967	0.9922	True	0.9951	0.9902	False	False	60.9084
40975	0.9949	0.9971	False	0.9783	0.9682	False	False	64.0157
40978	0.9790	0.9802	False	0.9886	0.9756	True	False	63.4900
40981	0.9081	0.8043	True	0.9189	0.7464	True	False	6.5967
40982	0.7878	0.7738	True	0.7685	0.7147	False	False	54.8814
40983	0.9819	0.9845	True	0.9935	0.9897	True	True	28.4800
40984	0.9395	0.9481	False	0.9216	0.9264	False	False	62.0967
40996	0.8971	0.9024	False	0.7740	0.7631	False	False	227.8870
41027	0.9007	0.8964	False	0.8190	0.8101	False	False	90.5901

Continued on next page

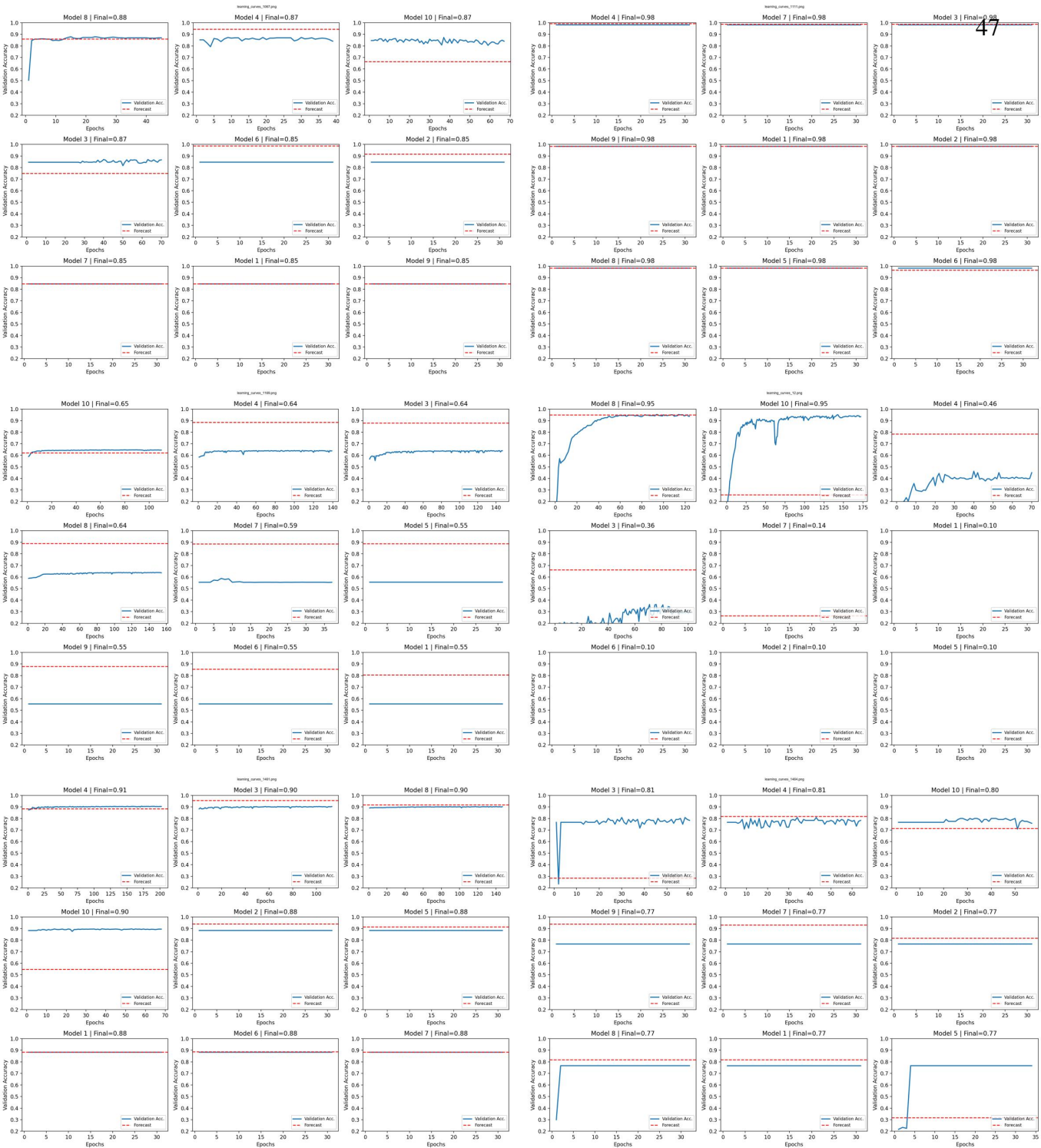
Table 6
Condensed EBE-NAS Experiment Results per Dataset

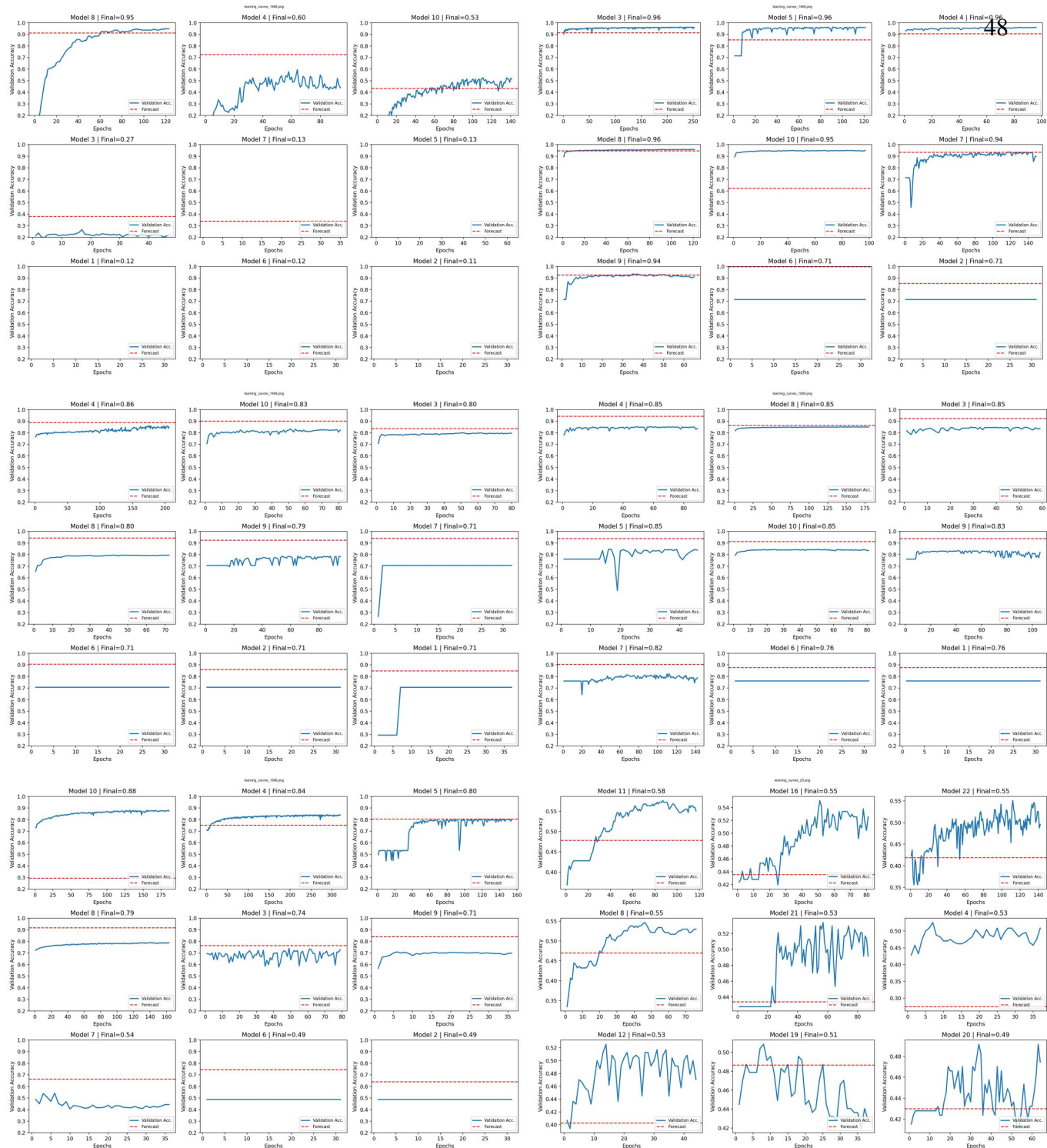
data_id	naml_best_validation_metric	naml_test_accuracy	ebe_decision	ebe_val_acc	ebe_test_acc	ebe_surpassed_val_baseline	ebe_surpassed_test_baseline	ebe_time_used_s
41138	0.9938	0.9934	False	0.9819	0.9819	False	False	90.7054
41142	0.7489	0.7565	True	0.7428	0.7223	False	False	73.4481
41143	0.8187	0.8291	True	0.8096	0.7672	False	False	39.6664
41144	0.8330	0.8551	True	0.6322	0.6178	False	False	61.4547
41145	0.7596	0.7686	True	0.7342	0.6847	False	False	52.0210
41146	0.9524	0.9424	False	0.8951	0.8985	False	False	65.6602
41147	0.6868	0.6853	False	0.5000	0.5000	False	False	143.3326
41150	0.9424	0.9424	False	0.9296	0.9255	False	False	95.5820
41156	0.8444	0.8554	True	0.8464	0.8470	True	False	59.7082
41157	0.8000	0.8000	True	0.8750	0.8000	True	True	3.0432
41158	0.9525	0.9398	False	0.9366	0.9144	False	False	64.2103
41159	0.8228	0.8320	False	0.6012	0.6000	False	False	119.5335
41161	0.9969	0.9972	False	0.9903	0.9922	False	False	144.7661
41162	0.8879	0.8858	True	0.8770	0.8770	False	False	90.5911
41163	0.9842	0.9860	False	0.9419	0.9415	False	False	160.3988
41164	0.7183	0.7130	False	0.6472	0.6292	False	False	65.7742
41165	0.5092	0.5320	True	0.3725	0.3380	False	False	1023.5956
41166	0.6788	0.6911	False	0.5175	0.5111	False	False	90.8297
41167	0.6986	0.5241	False	0.0059	0.0059	False	False	320.6525
41168	0.7105	0.7195	True	0.7082	0.7053	False	False	90.6231
41169	0.3751	0.3723	True	0.0729	0.0713	False	False	93.1162
42732	0.8784	0.8784	False	0.8784	0.8784	True	True	142.6315
42733	0.8360	0.8358	True	0.8317	0.8315	False	False	77.5396
42734	0.7500	0.7491	True	0.7158	0.7157	False	False	90.7542

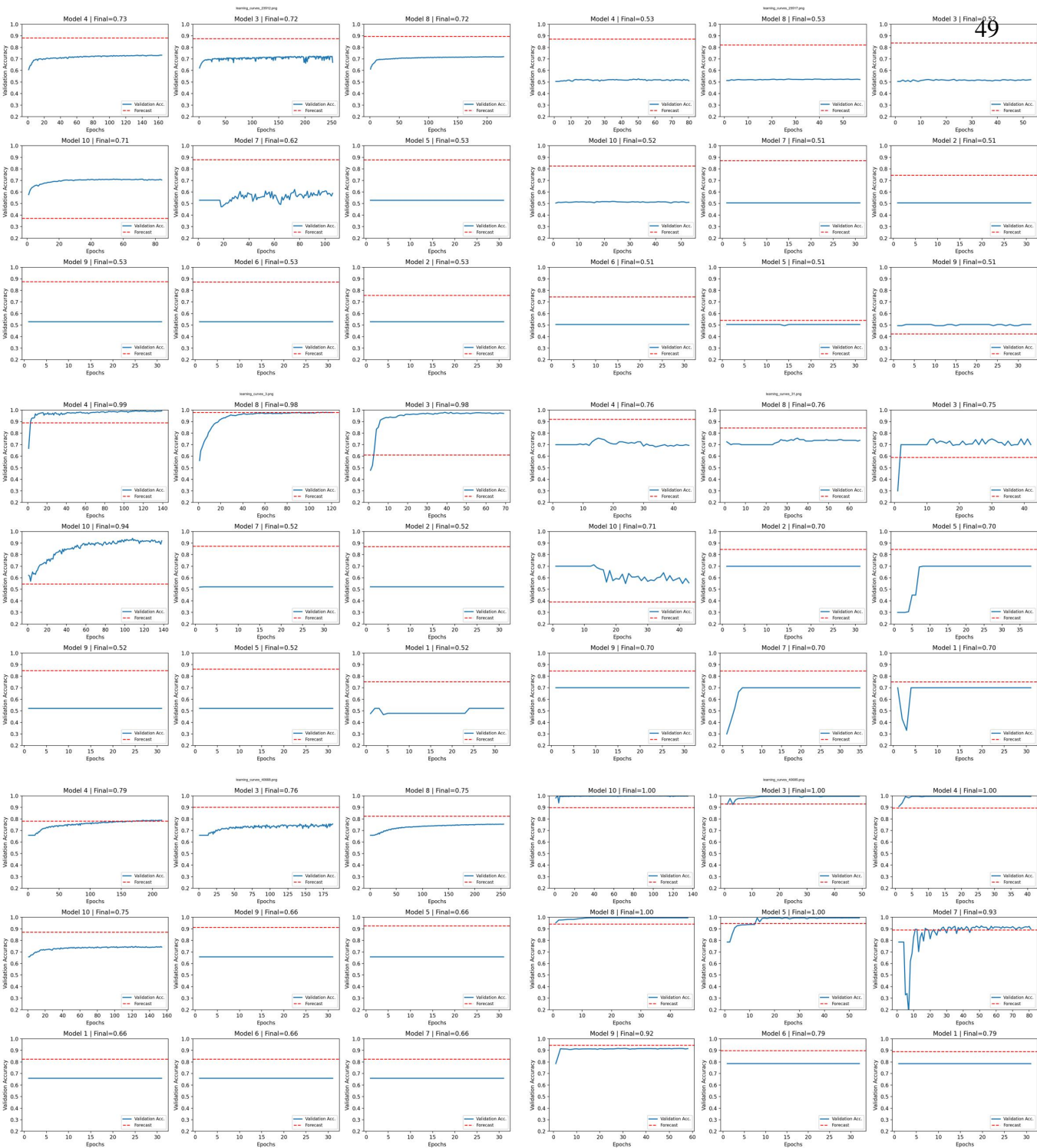
Continued on next page

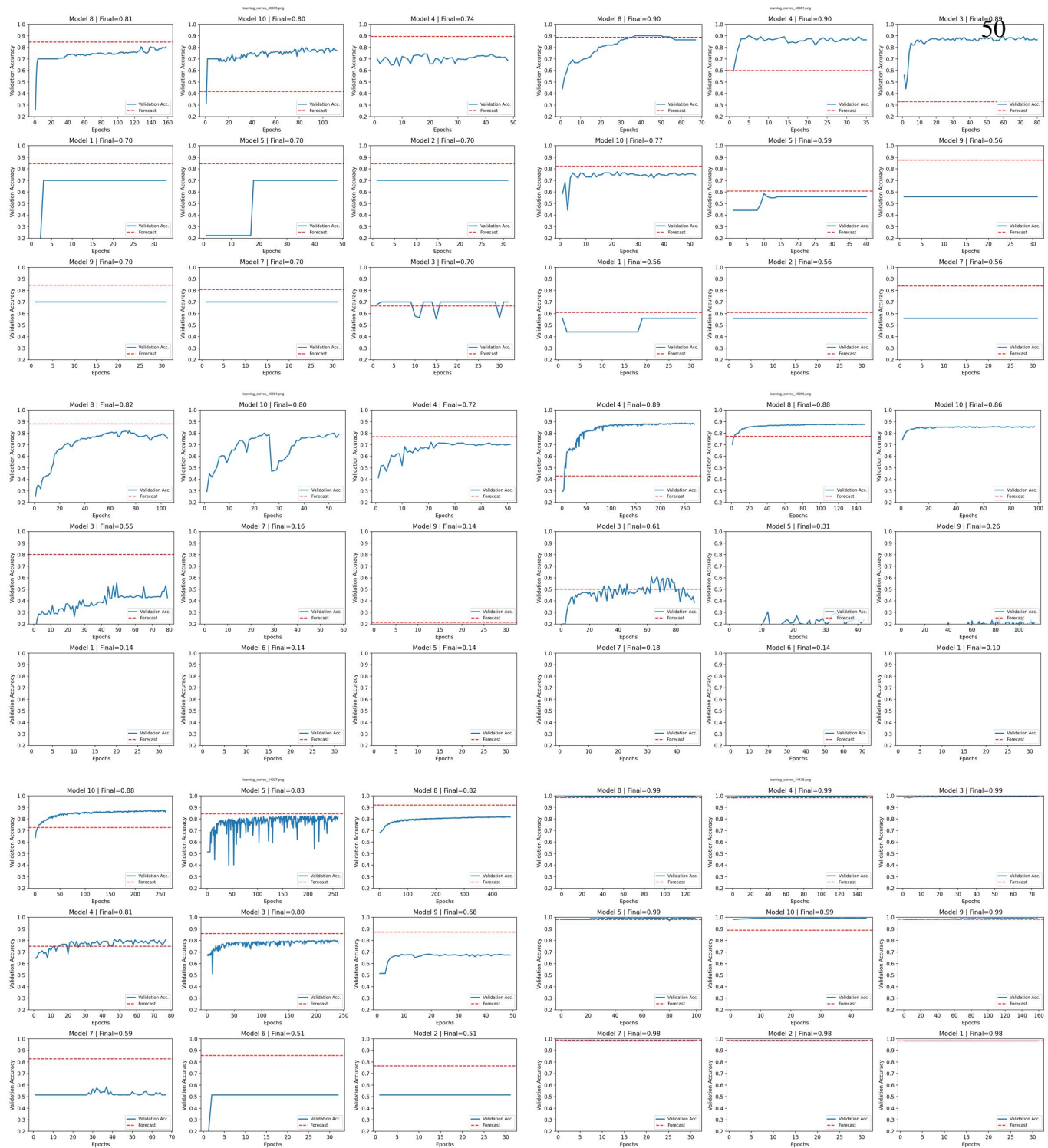
Table 6*Condensed EBE-NAS Experiment Results per Dataset*

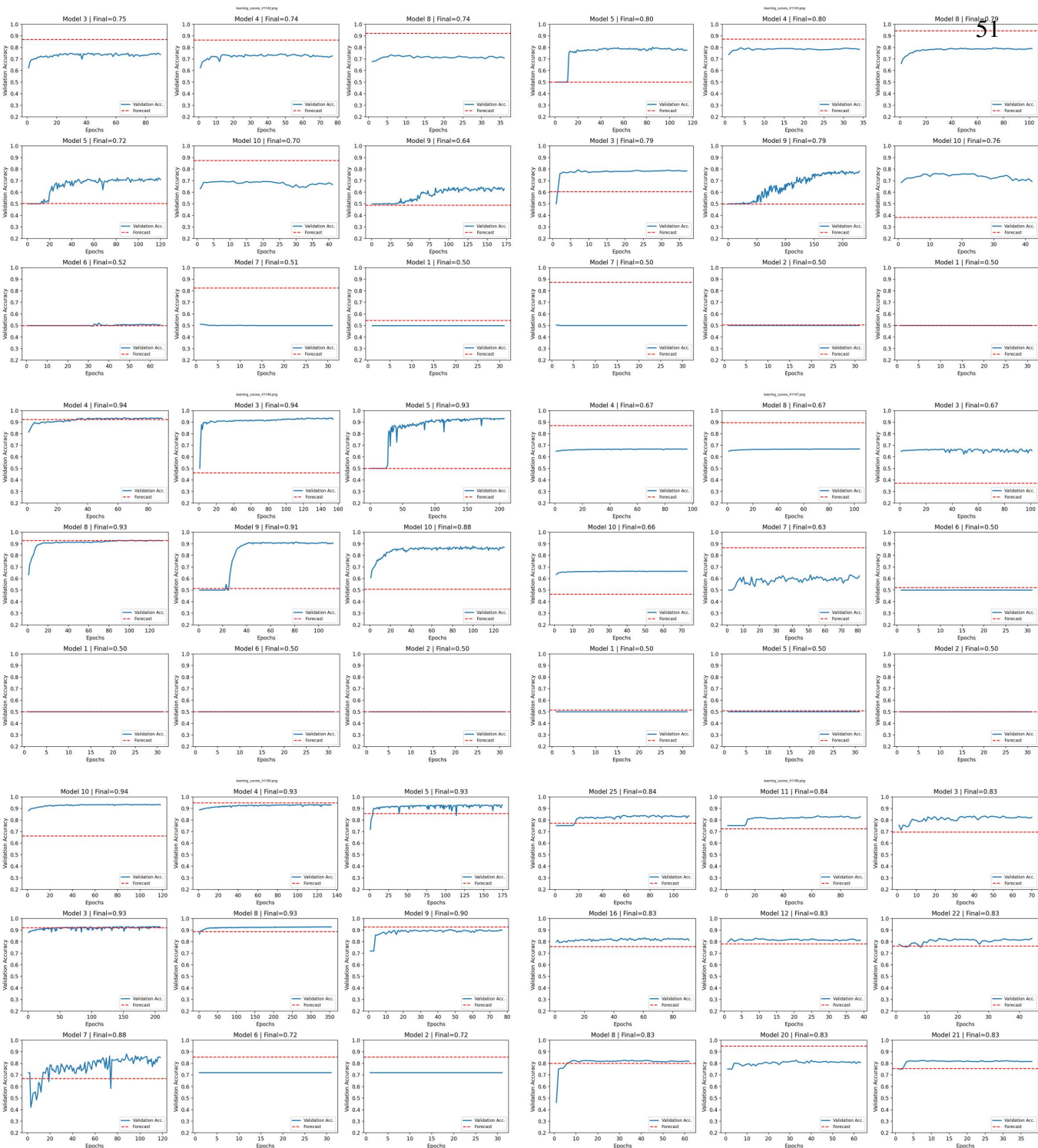
data_id	naml_best_validation_metric	naml_test_accuracy	ebe_decision	ebe_val_acc	ebe_test_acc	ebe_surpassed_val_baseline	ebe_surpassed_test_baseline	ebe_time_used_s
42742	0.9636	0.9636	False	0.9636	0.9636	False	True	117.7782
42746	1.0000	1.0000	False	0.5732	0.5732	False	False	207.5776
42769	0.7344	0.7355	False	0.5299	0.5299	False	False	208.1121

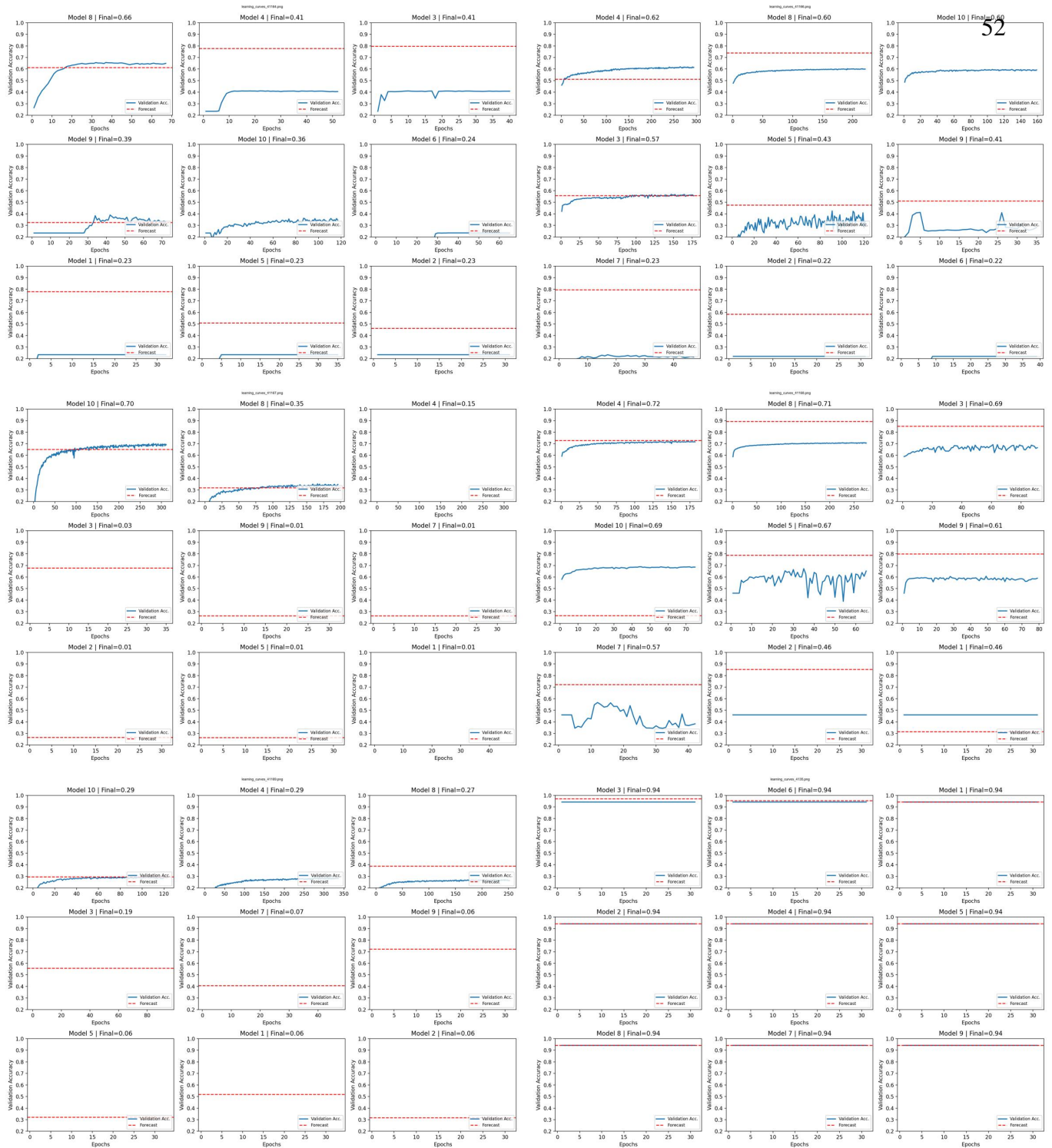


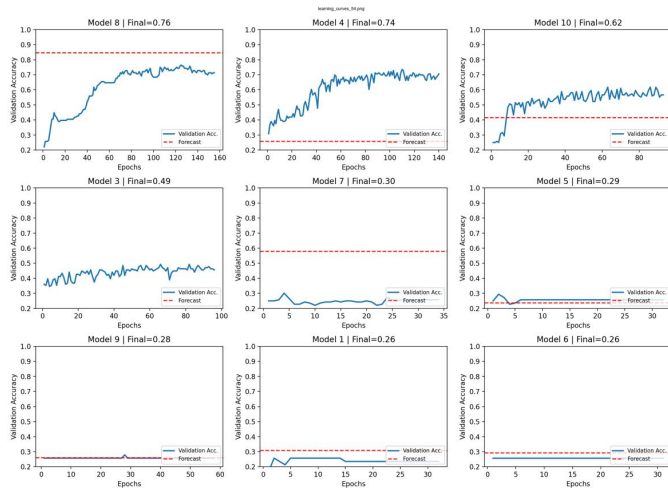




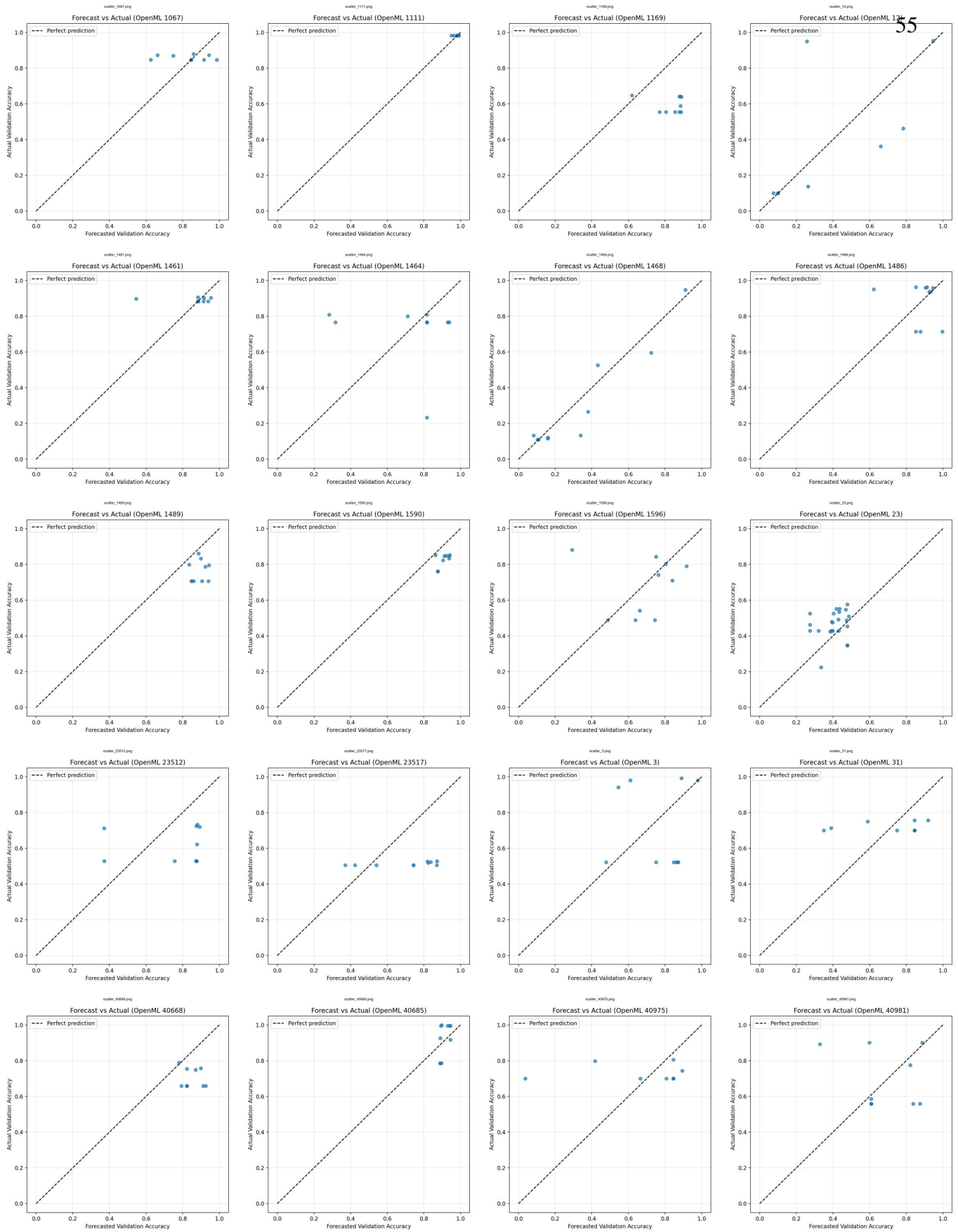


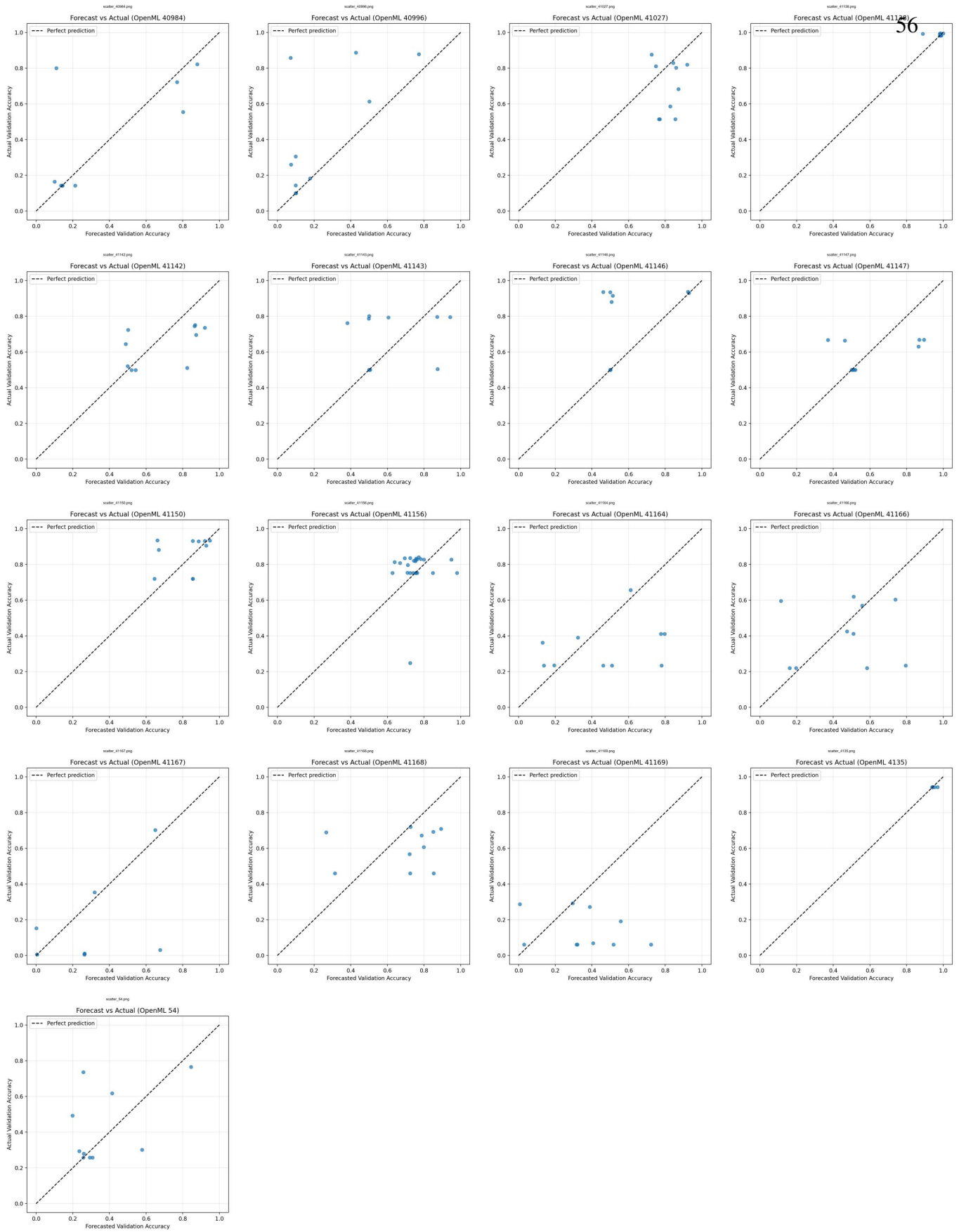




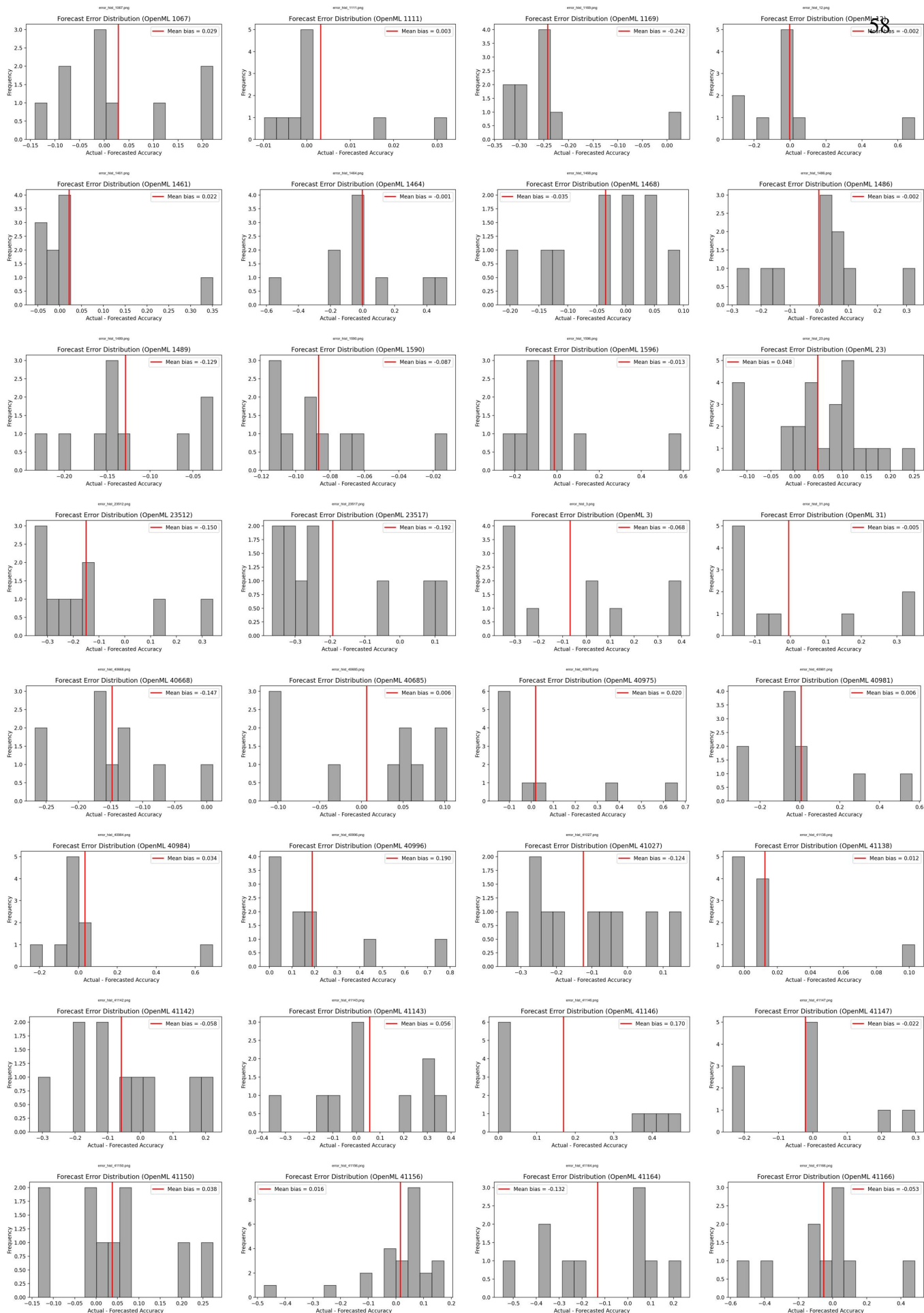


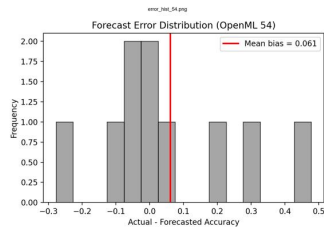
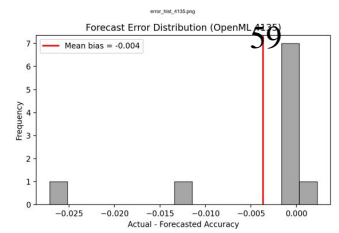
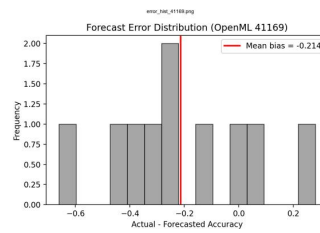
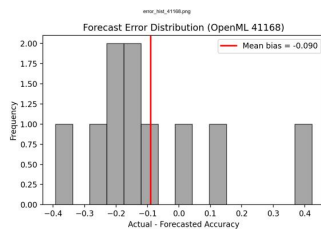
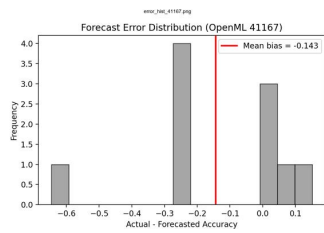
Appendix B: Scatterplot of the Forecast Performance





Appendix C: Error Histogram of the Forecast performance





Appendix D: ShouldYouEvenNN Generational Dynamics

